

جامعة دمشق

كلية الهندسة المعلوماتية

قسم ذكاء صناعي

# نظام التجارة الإلكترونية

(دراسة وتنفيذ المتطلبات غير الوظيفية)

إعداد الطلاب:

سفيان الثوري الكيلاني محمد الحلبي

إيمان عركور تسنيم مرعي

إشراف المهندس : حذيفة الخليل

السنة الجامعية

2026 - 2025

## الفهرس

| الرقم | العنوان                                                                                             |
|-------|-----------------------------------------------------------------------------------------------------|
| 1     | المتطلب غير الوظيفي الأول: حماية البيانات المشتركة من التضارب<br>Concurrent Access & Data Integrity |
| 1.1   | الجزء الأول: شرح عام عن المشكلة                                                                     |
| 1.1.1 | ما هي مشكلة التضارب Race Condition ؟                                                                |
| 1.2   | مصطلحات أساسية                                                                                      |
| 1.3   | الجزء الثاني: شرح المشكلة في المشروع                                                                |
| 1.4   | الجزء الثالث: استعراض الحلول الممكنة                                                                |
| 1.4.1 | الحل الأول: العمليات الذرية باستخدام Interlocked                                                    |
| 1.4.2 | الحل الثاني: قفل الاستبعاد المتبادل lock / Monitor                                                  |
| 1.4.3 | الحل الثالث: قوائم الانتظار الآمنة للخيوط Concurrent Collections                                    |
| 1.4.4 | الحل الرابع: التزامن المتفائل باستخدام RowVersion                                                   |
| 1.4.5 | الحل الخامس: SELECT FOR UPDATE و SKIP LOCKED في MySQL                                               |
| 1.4.6 | الحل السادس: الأقفال الموزعة باستخدام Redis / RedLock                                               |
| 1.4.7 | الحل السابع: تقسيم البيانات Sharding / Partitioning                                                 |
| 1.5   | الجزء الرابع: مقارنة شاملة لجميع الحلول                                                             |
| 1.6   | الجزء الخامس: الحل الأنسب لمشروعنا مع الأسباب                                                       |
| 1.7   | الجزء السادس: التطبيق العملي                                                                        |
| 1.8   | الجزء السابع: طريقة الاختبار قبل وبعد الحل                                                          |
| 2     | المتطلب غير الوظيفي الثاني: إدارة الموارد الحاسوبية<br>Resource Management & Capacity Control       |
| 2.1   | الجزء الأول: شرح عام عن المشكلة                                                                     |
| 2.2   | المصطلحات الأساسية                                                                                  |

|       |                                                                               |
|-------|-------------------------------------------------------------------------------|
| 2.3   | الجزء الثاني: شرح المشكلة في المشروع                                          |
| 2.4   | الجزء الثالث: الحلول الممكنة                                                  |
| 2.4.1 | الحل الأول: تقييد عدد الخيوط المتزامنة باستخدام SemaphoreSlim                 |
| 2.4.2 | الحل الثاني: تحديد حجم Thread Pool                                            |
| 2.4.3 | الحل الثالث: Rate Limiting                                                    |
| 2.4.4 | الحل الرابع: استخدام Message Queue                                            |
| 2.4.5 | الحل الخامس: تقييد عدد الاتصالات المتزامنة لقاعدة البيانات                    |
| 2.5   | الجزء الرابع: مقارنة الحلول                                                   |
| 2.6   | الجزء الخامس: الحل الأنسب لمشروعنا مع الأسباب                                 |
| 2.7   | التنفيذ العملي في المشروع خدمة IConcurrencyManager ومنهجية الاختبار           |
| 3     | المتطلب غير الوظيفي الثالث: المعالجة غير المتزامنة Asynchronous Queues        |
| 3.1   | الجزء الأول: شرح عام عن المفهوم                                               |
| 3.2   | المصطلحات الأساسية                                                            |
| 3.3   | الجزء الثاني: شرح المشكلة في المشروع                                          |
| 3.4   | الجزء الثالث: الحلول الممكنة                                                  |
| 3.4.1 | الحل الأول: استخدام Fire-and-Forget مع Task.Run                               |
| 3.4.2 | الحل الثاني: استخدام Channel<T> مع BackgroundService                          |
| 3.4.3 | الحل الثالث: استخدام Hangfire                                                 |
| 3.4.4 | الحل الرابع: استخدام ConcurrentQueue<T> مع BackgroundService                  |
| 3.5   | الجزء الرابع: مقارنة الحلول                                                   |
| 3.6   | الجزء الخامس: الحل الأنسب للمشروع                                             |
| 3.7   | الجزء السادس: التطبيق العملي باستخدام Hangfire                                |
| 3.8   | الجزء السابع: اختبار الحل وإثباته                                             |
| 4     | المتطلب غير الوظيفي الرابع: معالجة البيانات الضخمة على دفعات Batch Processing |
| 4.1   | الجزء الأول: شرح عام عن المشكلة                                               |

|     |                                                             |
|-----|-------------------------------------------------------------|
| 4.2 | المصطلحات الأساسية                                          |
| 4.3 | الجزء الثاني: شرح المشكلة في مشروعنا                        |
| 4.4 | الجزء الثالث: الحلول الممكنة                                |
| 4.5 | الجزء الرابع: مقارنة الحلول                                 |
| 4.6 | الجزء الخامس: الحل الأنسب لمشروعنا مع الأسباب               |
| 4.7 | الجزء السادس: التطبيق العملي                                |
| 4.8 | اختبارات وإثبات Batch Processing على التقارير وعرض المنتجات |
| 5   | المتطلب غير الوظيفي الخامس: توزيع الأحمال Load Distribution |
| 5.1 | الجزء الأول: شرح عام عن المفهوم                             |
| 5.2 | الجزء الثاني: شرح المشكلة في مشروعنا                        |
| 5.3 | الجزء الثالث: الحلول الممكنة لمحاكاة توزيع الأحمال          |
| 5.4 | الجزء الرابع: مقارنة الحلول                                 |
| 5.5 | الجزء الخامس: الحل الأنسب للمشروع                           |
| 5.6 | الجزء السادس: التطبيق العملي - إعداد YARP                   |
| 5.7 | الجزء السابع: اختبار توزيع الأحمال                          |
| 6   | المراجع                                                     |

## المتطلب غير الوظيفي الأول:

### حماية البيانات المشتركة من التضارب

## Concurrent Access & Data Integrity

الجزء الأول: شرح عام عن المشكلة

ما هي مشكلة التضارب **Race Condition** ؟

في عالم البرمجة المتوازية، عندما يقوم برنامجان أو خيطان **Threads** بالوصول إلى نفس البيانات في نفس الوقت، قد تحدث حالة تسمى حالة السباق **Race Condition** تخيل أن لدينا صندوقاً يحتوي على 5 تقاحات، وشخصين يريدان شراء التفاح في نفس اللحظة. النظام الذي لا يحمي البيانات قد يسمح لكل منهما بشراء آخر تفاحة، فيصبح المخزون -1 ، وهذا غير منطقي في الواقع.

في أنظمة التجارة الإلكترونية، هذه المشكلة خطيرة جداً. عندما يضغط 100 مستخدم على زر شراء في نفس الوقت، يجب أن يضمن النظام أن المستخدمين الذين يسبقون غيرهم يحصلون على المنتج، وأن الآخرين يعرفون أن المنتج نفذ.

لماذا تحدث هذه المشكلة؟

تحدث حالة السباق لسببين رئيسيين:

**زمن الاستجابة:** قراءة البيانات من قاعدة البيانات وتحديثها تستغرق وقتاً (حتى لو كان أجزاء من الألف من الثانية). في هذا الوقت، يمكن لعمليات أخرى أن تقرأ القيمة القديمة.

**التنفيذ المتوازي:** تطبيقات الويب الحديثة تتعامل مع كل طلب في خيط منفصل، وهذه الخيوط تعمل بالتوازي على نفس المعالج أو على معالجات مختلفة.

| المصطلح                       | المعنى التقني                                                                                                      | المعنى العملي                                                |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Race Condition</b>         | حالة يتنافس فيها خيطان أو أكثر على تعديل نفس البيانات، وتكون النتيجة النهائية غير متوقعة وتعتمد على ترتيب تنفيذهما | مستخدمان يشتريان آخر قطعة في نفس الوقت، كلاهما يعتقد أنه نجح |
| <b>Lost Update</b>            | نوع من Race Condition حيث تفقد عملية تحديث أثرها بسبب تداخل عملية تحديث أخرى                                       | تحديث المخزون الأول يضيع، ويصبح المخزون غير دقيق             |
| <b>Optimistic Concurrency</b> | استراتيجية تفترض أن التضارب نادر، وتسمح بالتنفيذ المتوازي، وتتحقق من عدم وجود تضارب قبل الحفظ                      | نتق أن التضارب لن يحدث، ولكن إذا حدث نتعامل معه              |
| <b>Pessimistic Locking</b>    | استراتيجية تقفل البيانات قبل استخدامها، وتمنع أي عملية أخرى من الوصول إليها حتى يتم الانتهاء                       | نغلق المنتج قبل استخدامه، لا يمكن لأحد لمسه حتى ننتهي        |
| <b>Distributed Lock</b>       | قفل يعمل عبر عدة خوادم في بيئة موزعة                                                                               | قفل يرى جميع الخوادم، حتى لو كان التطبيق يعمل على 10 أجهزة   |

## الجزء الثاني: شرح المشكلة في المشروع

### سيناريو المشكلة في النظام

في مشروعنا، لدينا عملية رئيسية هي "شراء منتج". هذه العملية تتكون من عدة خطوات:

- الخطوة 1: قراءة المخزون الحالي من قاعدة البيانات
- الخطوة 2: التحقق من أن الكمية المطلوبة متوفرة
- الخطوة 3: خفض المخزون (المخزون = المخزون - الكمية)
- الخطوة 4: حفظ التغيير في قاعدة البيانات
- الخطوة 5: إنشاء سجل الطلب
- الخطوة 6: معالجة الدفع

### المشكلة بالتفصيل (سيناريو واقعي):

لنفترض أن لدينا منتجاً يتوفر منه قطعة واحدة فقط. في نفس الثانية، يرسل مستخدمان A و B طلب شراء.

في نظام غير محمي بدون Concurrency Control ، سيحدث التالي:

| الوقت       | الخيطة | الحدث                             | المخزون في قاعدة البيانات |
|-------------|--------|-----------------------------------|---------------------------|
| <b>T0</b>   | A      | يقرأ المخزون                      | 1                         |
| <b>T0.1</b> | B      | يقرأ المخزون ( لم يكتب A بعد )    | 1                         |
| <b>T1</b>   | A      | يحسب : $1 - 1 = 0$                | 1                         |
| <b>T1.1</b> | B      | يحسب : $1 - 1 = 0$                | 1                         |
| <b>T2</b>   | A      | يكتب المخزون الجديد = 0           | 0                         |
| <b>T2.1</b> | B      | يكتب المخزون الجديد = 0           | 0                         |
|             |        | خطأ: كان يجب أن يصبح 1- أو يرفض B |                           |
|             |        | فوق قيمة A                        |                           |

### النتيجة:

تم قبول عمليتي شراء لقطعة واحدة فقط!

تحديث الخيط A (المستخدم الأول) ضاع (Lost Update)

المخزون النهائي = 0 (صحيح)، ولكن تم بيع قطعة غير موجودة!

المستخدم الثاني سيدفع ثمناً لمنتج لن يحصل عليه

### الجزء الثالث: استعراض الحلول الممكنة

قبل أن نختار الحل الأنسب، يجب أن نعرف كل الأدوات المتاحة في .NET. لحماية البيانات من التضارب.

سأقدم كل حل مع شرح مبسط ومتى نستخدمه.

## الحل الأول: العمليات الذرية (Atomic Operations) باستخدام Interlocked:

ما هي المشكلة التي يحلها؟

عندما تكون العملية بسيطة مثل زيادة عداد counter++ هذه العملية تتحلل إلى ثلاث خطوات على مستوى المعالج:

قراءة، تعديل، كتابة. إذا توقف الخيط بين هذه الخطوات، يحدث تضارب.

### كيف يعمل Interlocked؟

توفر .NET كلاس Interlocked الذي يضمن أن هذه العمليات الثلاث تنفذ ككتلة واحدة غير قابلة للمقاطعة على مستوى الأجهزة، باستخدام تعليمات المعالج

```
int counter = 0;
// غير آمن في بيئة متعددة الخيوط
counter++;
// آمن وذري (Atomic)
Interlocked.Increment(ref counter);
```

ما هي العمليات التي يوفرها Interlocked؟

| الوظيفة                                               | الدالة                             |
|-------------------------------------------------------|------------------------------------|
| زيادة العداد بمقدار 1                                 | Increment(ref int)                 |
| نقصان العداد بمقدار 1                                 | Decrement(ref int)                 |
| إضافة قيمة معينة                                      | Add(ref int, int)                  |
| استبدال القيمة بأخرى                                  | Exchange(ref int, int)             |
| مقارنة وتبادل (إذا كانت القيمة الحالية مساوية لمتوقع) | CompareExchange(ref int, int, int) |



## 1. عداد المستخدمين المتصلين (Active Users Counter) :

```
public static class ActiveUsersCounter
{
    private static int _activeUsers = 0;

    public static void Increment() => Interlocked.Increment(ref
_activeUsers);
    public static void Decrement() => Interlocked.Decrement(ref
_activeUsers);
    // قراءة ذرية
    public static int GetCount() => Interlocked.CompareExchange(ref
_activeUsers, 0, 0);
}
تجميع إحصائيات الطلبات.2.
private static long _totalOrders = 0;
private static long _totalSalesAmount = 0;

public static void AddOrder(decimal amount)
{
    Interlocked.Increment(ref _totalOrders);
    Interlocked.Add(ref _totalSalesAmount, (long)amount);
}
```

متى نستخدم **Interlocked**؟

| مناسب                                                     | غير مناسب                                        |
|-----------------------------------------------------------|--------------------------------------------------|
| العملية بسيطة (زيادة، نقصان، جمع)                         | لعملية مركبة (مثل التحقق من الرصيد ثم خصمه)      |
| التزامن عالٍ جداً آلاف العمليات في الثانية                | نحتاج إلى قراءة حالة معقدة واتخاذ قرار           |
| لا نحتاج إلى التراجع (Rollback) أو معاملات متعددة الجداول | هناك حاجة إلى التراجع عن التغييرات إذا فشلت خطوة |

لماذا لا نستخدم **Interlocked** لتحديث المخزون؟

تحديث المخزون ليس مجرد زيادة أو نقصان بسيط. يجب:

- قراءة الرصيد الحالي
- التحقق من أن الرصيد كافٍ
- خصم الكمية
- تسجيل حركة المخزون
- إنشاء أمر الشراء
- التراجع عن كل شيء إذا فشل الدفع

Interlocked لا يستطيع تنفيذ كل هذه الخطوات كعملية واحدة ذرية.

**خلاصة:** نستخدم Interlocked فقط للإحصائيات والعدادات الخفيفة. عمليات المخزون والدفع تحتاج إلى طرق أخرى.

## الحل الثاني: قفل الاستبعاد المتبادل (lock / Monitor)

ما هي المشكلة التي يحلها؟

عندما نحتاج إلى تنفيذ مجموعة متكاملة من التعليمات البرمجية (عدة أسطر) بشكل ذري، لا تكفي العمليات الذرية البسيطة. نحتاج إلى آلية تضمن أن خيطاً واحداً فقط يمكنه تنفيذ هذه المجموعة في أي لحظة.

كيف يعمل lock في C# ؟

يتطلب lock كائن قفل (عادة كائن خاص). الخيط الذي يصل أولاً يطلب القفل ويدخل المنطقة الحرجة

(Critical Section) أي خيط آخر يصل لاحقاً ينتظر في طابور حتى يتم تحرير القفل.

```
private readonly object _lockObject = new object();
private readonly object _lockObject = new object();

public void DoSomethingCritical()
{
    lock (_lockObject)
    {
        // منطقة حرجة: خيط واحد فقط يدخل هنا في كل مرة
    }
}
```

```

        // قراءة وكتابة البيانات المشتركة
    }
}

```

### ماذا يحدث خلف الكواليس؟

يقوم المترجم بتحويل كتلة lock إلى استخدام Monitor.Enter و Monitor.Exit مع كتلة try/finally لضمان تحرير القفل حتى في حالة حدوث استثناء.

```

object __lockObj = _lockObject; object __lockObj = _lockObject;
bool __lockTaken = false;
try
{
    Monitor.Enter(__lockObj, ref __lockTaken);
    // المنطقة الحرجة
}
finally
{
    if (__lockTaken) Monitor.Exit(__lockObj);
}

```

### تطبيقات عملية في المشروع

#### 1. تحديث سلة تسوق المستخدم:

عندما يفتح المستخدم عدة علامات تبويب في المتصفح، قد تأتي عدة طلبات لتعديل نفس السلة في نفس الوقت

```

public class ShoppingCart

    private readonly object _cartLock = new object();
    private Dictionary<int, int> _items = new Dictionary<int, int>();

    public void AddItem(int productId, int quantity)
    {
        lock (_cartLock)
        {
            if (_items.ContainsKey(productId))
                _items[productId] += quantity;
            else
                _items[productId] = quantity;
        }
    }
}

```

```

public void RemoveItem(int productId)
{
    lock (_cartLock)
    {
        _items.Remove(productId);
    }
}

```

متى نستخدم lock؟

| مناسب                                                                  | غير مناسب                                                       |
|------------------------------------------------------------------------|-----------------------------------------------------------------|
| نريد تنفيذ مجموعة من العمليات بشكل ذري<br>(عدة قراءات وكتابات)         | نحتاج فقط إلى زيادة عداد استخدم Interlocked                     |
| العملية سريعة نسبياً (أقل من ميلي ثانية)<br>لا تتضمن I/O أو اتصال شبكة | العملية تتضمن عدة كائنات قفل قد تؤدي إلى مأزق<br>(Deadlock)     |
| نتعامل مع بيانات في الذاكرة فقط (وليس قاعدة بيانات)                    | المنطقة الحرجة طويلة (أكثر من بضع ميلي ثانية) ستسبب<br>اختناقاً |

### عيوب lock في البيئة الموزعة

**العيوب الأكبر lock** : يعمل فقط داخل نفس العملية (Process) ونفس الخادم. إذا كان التطبيق يعمل على عدة خوادم، فإن قفل الخادم الأول لا يؤثر على الخادم الثاني، وقد يحدث تضارب بين الخوادم.

### الحل الثالث: قوائم الانتظار الآمنة للخيوط (Concurrent Collections)

ما هي المشكلة التي تحلها؟

المجموعات التقليدية مثل `List<T>` و `Dictionary<K,V>` غير آمنة للاستخدام في بيئة متعددة الخيوط. إذا حاول خيطان إضافة عنصر إلى `List<T>` في نفس الوقت، قد تتلف البنية الداخلية للقائمة.

**ماذا تقدم Concurrent Collections؟** قدمت Microsoft مساحة أسماء `System.Collections.Concurrent` التي تحتوي على هياكل بيانات آمنة للخيوط (thread-safe) مصممة خصيصاً للوصول المتزامن من عدة خيوط.

| المجموعة                                       | الوظيفة                                     | التناظر غير الآمن          |
|------------------------------------------------|---------------------------------------------|----------------------------|
| <b>ConcurrentQueue&lt;T&gt;</b>                | طابور FIFO<br>First In First Out            | Queue<T>                   |
| <b>ConcurrentStack&lt;T&gt;</b>                | مكدس LIFO<br>Last In First Out              | Stack<T>                   |
| <b>ConcurrentDictionary&lt;TKey,TValue&gt;</b> | قاموس آمن للخيوط                            | Dictionary<TKey,TValue>    |
| <b>BlockingCollection&lt;T&gt;</b>             | عازل مع إمكانية الحظر<br>والحد الأقصى للسعة | Queue<T> مع AutoResetEvent |

### كيف تعمل Concurrent Collections ؟

بدلاً من قفل المجموعة بأكملها، تستخدم هذه المجموعات تقنيات متقدمة:

**ConcurrentQueue<T>**: تستخدم مصفوفة من المقاطع Segments وخوارزمية Lock-Free تعتمد على مبادلة

الرؤوس والذيل ذرياً. الخيوط المنتجة تتنافس فقط على مؤشر الذيل، والخيوط المستهلكة على مؤشر الرأس.

**ConcurrentDictionary<TKey,TValue>**: تقسم الجدول الداخلي إلى عدة أقفال (حسب عدد مراحل التجزئة). تحديث

المفاتيح المختلفة قد لا تتنافس على القفل نفسه.

**ConcurrentBag<T>**: تحتفظ بقائمة محلية لكل خيط، وتقوم بـ "سرقة" العمل من خيوط أخرى عند الحاجة

### تطبيقات عملية في المشروع

#### 1. طابور الطلبات غير المعالجة (Order Queue) :

```
public static class OrderQueue
{
    private static readonly ConcurrentQueue<OrderRequest> _queue = new();

    public static void Enqueue(OrderRequest request) => _queue.Enqueue(request);

    public static bool TryDequeue(out OrderRequest request) => _queue.TryDequeue(out request);
}
```

## 2. طابور سجل الأحداث (Event Log Queue) :

```
private static readonly ConcurrentQueue<LogEvent> _eventQueue = new();

public static void LogEvent(string message)
{
    _eventQueue.Enqueue(new LogEvent(message, DateTime.UtcNow));
}
```

### متى نستخدم Concurrent Collections ؟

| السبب                   | الموصى بها                | السيناريو                       |
|-------------------------|---------------------------|---------------------------------|
| أداء عالٍ، Lock-Free    | ConcurrentQueue<T>        | طابور منتج/مستهلك بسيط          |
| أقفال دقيقة، توازن جيد  | ConcurrentDictionary<K,V> | قاموس للـ Cache من عدة خيوط     |
| يقلل التصارع بين الخيوط | ConcurrentBag<T>          | كل خيط له مجموعة خاصة به        |
| يتحكم بتدفق الإنتاج     | BlockingCollection<T>     | طابور بعازل وحظر (Backpressure) |

### متى لا نستخدم Concurrent Collections ؟

- المجموعة صغيرة والتزامن معدوم lock على List<T> أبسط
- نحتاج معاملات ذرية متعددة العمليات (انقل عنصراً من طابور إلى آخر يجب أن يكون ذرياً) → تحتاج قفل أعلى مستوى
- نحتاج ترتيباً صارماً غير FIFO/LIFO
- العمليات في الغالب قراءة فقط نادرة والكتابة كثيرة

## الحل الرابع: التزامن المتفائل (Optimistic Concurrency) باستخدام RowVersion

ما هي فلسفة التزامن المتفائل؟

في عالم قواعد البيانات المتعددة المستخدمين، توجد فلسفتان متعارضتان:

**التزامن المتشائم Pessimistic**: الخطر محقق، فلنغلق كل شيء.

عند قراءة صف بيانات، يتم قفله حتى نهاية المعاملة. هذا يضمن عدم تعديله من قبل آخرين، ولكنه يخلق اختناقات.

**التزامن المتفائل Optimistic**: لنفترض أن التضارب نادر.

لا يتم قفل الصف عند القراءة. بدلاً من ذلك، عند التحديث، نتحقق مما إذا كان الصف قد تغير منذ قراءته.

لماذا نختار التزامن المتفائل في التجارة الإلكترونية؟

في أنظمة التجارة الإلكترونية:

- عمليات القراءة (عرض المنتجات) تفوق عمليات الكتابة (الشراء) بنسبة كبيرة جداً (مثلاً 1:1000)
- نافذة التضارب على نفس المنتج ضيقة جداً (ثوانٍ قليلة)
- المستخدمون مستعدون لإعادة المحاولة إذا فشل طلبهم بسبب التضارب

**الآلية: كيف يعمل RowVersion؟**

تعتمد الآلية على وجود عمود خاص RowVersion يعمل كـ "رقم إصدار" للصف. كلما تم تحديث الصف، يتغير هذا العمود تلقائياً.

**في SQL Server :**

```
CREATE TABLE Products (
  Id INT PRIMARY KEY,
  Name NVARCHAR(200),
  Stock INT,
  RowVersion ROWVERSION
  -- يتغير تلقائياً عند أي تحديث
);
```

## في كيان Product :

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Stock { get; set; }

    [Timestamp] // EF Core يخبر
    public byte[] RowVersion { get; set; }
}
```

## أثناء التحديث:

```
UPDATE Products
SET Stock = @newStock
WHERE Id = @id AND RowVersion = @oldRowVersion
```

إذا قامت معاملة أخرى بتحديث نفس الصف بعد قراءتنا، فإن RowVersion في قاعدة البيانات سيكون مختلفاً، ولن يجد جملة WHERE أي صف لتحديثه. قاعدة البيانات ترجع 0 rows affected ، ويرمي EF Core استثناء DbUpdateConcurrencyException

## متى يكون التزامن المتفائل مناسباً؟

| مناسب                                             | غير مناسب                                       |
|---------------------------------------------------|-------------------------------------------------|
| نسبة القراءة إلى الكتابة عالية جداً               | تضارب متكرر وعالي (مثل حجز تذاكر حفلة جماهيرية) |
| فرصة التضارب منخفضة                               | معاملات طويلة (المستخدم يعدل بيانات لعدة دقائق) |
| تطبيقات عديمة الحالة (Stateless) مثل APIs الحديثة | تطبيقات تتطلب تنبيهات مباشرة بالتضارب           |
| بيئة موزعة (Horizontal Scaling)                   |                                                 |



## الحل الخامس: SELECT FOR UPDATE و SKIP LOCKED في MySQL

ما هي الفكرة؟

هذا حل على مستوى قاعدة البيانات يستخدم القفل المتشائم Pessimistic Locking عند قراءة صف ما، نطلب قفله (FOR UPDATE) حتى ننتهي من المعاملة. لا يمكن لأي معاملة أخرى قراءة أو تعديل هذا الصف.

### مشكلة FOR UPDATE التقليدية

في الإصدارات القديمة، إذا كان هناك قفل على صف ما، فإن المعاملات الأخرى تنتظر (Block) حتى يتم تحرير القفل. هذا يخلق اختناقات وطوابير انتظار.

**SKIP LOCKED** تحل هذه المشكلة: إذا كان الصف مقفلاً بواسطة معاملة أخرى، يتم تخطيه وإرجاع الصفوف المتاحة فقط. هذا يحول جدول المنتج إلى قائمة انتظار موزعة عالية الكفاءة.

### التطبيق العملي في MySQL

```
SELECT * FROM Products WHERE Id = 1 FOR UPDATE SKIP LOCKED;
```

### التطبيق في .NET مع SQL

```
var sql = "SELECT * FROM Products WHERE Id = @productId FOR UPDATE SKIP LOCKED";  
var product = await _context.Products  
    .FromSqlRaw(sql, new SqlParameter("@productId", productId))  
    .FirstOrDefaultAsync();
```

متى نستخدم هذا الحل

| مناسب                                      | غير مناسب                                    |
|--------------------------------------------|----------------------------------------------|
| نستخدم MySQL 8.0.1 أو أحدث                 | إصدار MySQL أقل من 8.0.1 لا يدعم SKIP LOCKED |
| نريد قفل وقائي (Pessimistic) بدون اختناقات | نفضل تجنب SQL الخادم في EF Core              |
| نريد تحويل جدول المنتج إلى قائمة انتظار    | نسبة التضارب منخفضة (التزامن المتقائل أفضل)  |

## الحل السادس: الأقفال الموزعة (Distributed Lock) باستخدام Redis / RedLock

ما هي المشكلة التي يحلها هذا الحل؟

في البيئة الموزعة (عدة خوادم)، lock في C# يعمل فقط على الخادم المحلي. الخادم A لا يرى القفل على الخادم B. هذا يعني أن عملية توزيع الأحمال (Load Balancing) قد تتسبب في تضارب بين الخوادم.

ما هو RedLock ؟

**RedLock:** هو خوارزمية قفل موزع اقترحها مبتكر Redis بدلاً من الاعتماد على خادم Redis واحد، يتعامل مع عدة خوادم مستقلة.

الخوارزمية:

- للحصول على القفل، يجب الحصول على موافقة من أغلبية الخوادم (مثل 3 من أصل 5)
- القفل له مهلة زمنية (TTL) تنتهي تلقائياً
- هذا يضمن عدم "سرقة" القفل حتى لو تعطل أحد الخوادم

التطبيق في .NET باستخدام RedLock.net

- تثبيت الحزمة:

```
dotnet add package RedLock.net
```

## ■ التسجيل في Program.cs :

```
var multiplexers = new List<RedLvar multiplexers = new List<RedLockMultiplexer>
{
    new RedLockMultiplexer(ConnectionMultiplexer.Connect("redis-node1:6379")),
    new RedLockMultiplexer(ConnectionMultiplexer.Connect("redis-node2:6379")),
    new RedLockMultiplexer(ConnectionMultiplexer.Connect("redis-node3:6379"))
};
builder.Services.AddSingleton<IDistributedLockFactory>(new
RedLockFactory(multiplexers));
```

## ■ الاستخدام:

```
await using var redLock = await _lockFactory.CreateLockAsync($"product:{productId}",
expiryawait using var redLock = await
_lockFactory.CreateLockAsync($"product:{productId}", expiry);
if (redLock.IsAcquired)
{
    // تم الحصول على القفل - تنفيذ العملية بأمان
    // حتى لو كان التطبيق يعمل على 10 خوادم مختلفة
}
else
{
    // فشل الحصول على القفل - المنتج قيد المعالجة في خادم آخر
}
```

## متى نستخدم RedLock ؟

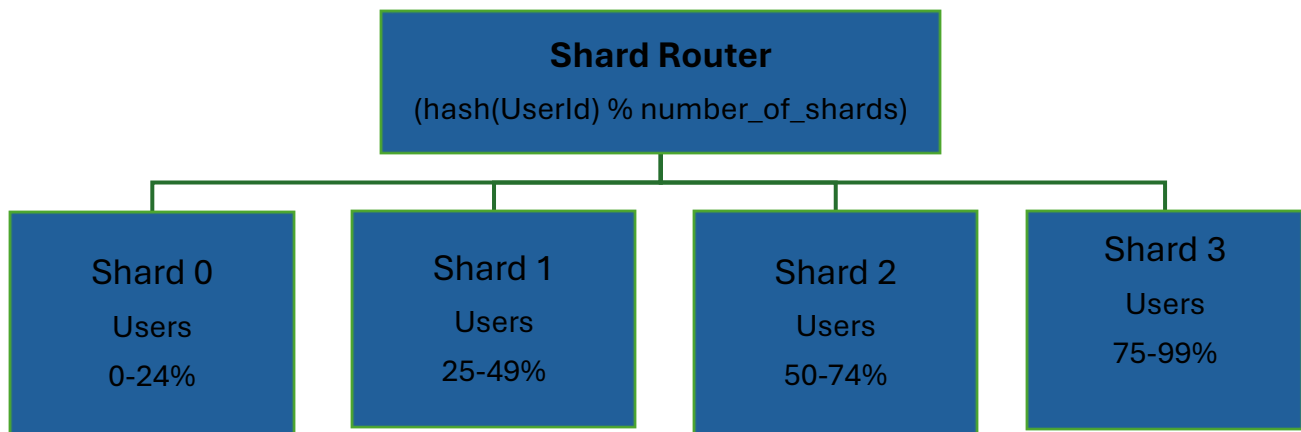
| مناسب                                                     | غير مناسب                                        |
|-----------------------------------------------------------|--------------------------------------------------|
| التطبيق يعمل على عدة خوادم (Horizontal Scaling)           | تطبيق خادم واحد overkill lock أو RowVersion يكفي |
| سلامة البيانات هي الأولوية المطلقة (مثل العمليات المالية) | حمولة خفيفة (إضافة تعقيد لا داعي له)             |
| لدينا بنية تحتية لخوادم Redis يفضل 3 أو 5                 | لا توجد بنية تحتية لـ Redis                      |

## الحل السابع: تقسيم البيانات Sharding / Partitioning

ما هي الفكرة الأساسية؟

بدلاً من محاولة جعل قاعدة بيانات واحدة تتحمل كل الضغط، نقسم البيانات الكبيرة إلى أجزاء أصغر مستقلة تسمى Shards. يتم توزيع هذه الأجزاء على خوادم منفصلة. المعاملات التي تخص مفاتيح تجزئة مختلفة تعمل بالتوازي الكامل ولا تتنافس على نفس الموارد.

كيف يعمل Sharding ؟



استراتيجيات Sharding

| الاستراتيجية | الآلية                                        | متى تستخدم                       |
|--------------|-----------------------------------------------|----------------------------------|
| Hash-based   | $\text{Shard} = \text{hash}(\text{Key}) \% N$ | توزيع متوازن، يمنع العقد الساخنة |
| Range-based  | تقسيم حسب نطاق القيم                          | استعلامات النطاق فعالة           |
| Time-based   | تقسيم حسب الوقت (شهر 1، شهر 2)                | أرشفة وحذف سريع                  |
| Entity-based | فصل الجداول الكبيرة في قواعد بيانات منفصلة    | أبسط أشكال التوزيع               |

```
public class ShardRouter
{
    private readonly int _numberOfShards = 4;
    private readonly string[] _connectionStrings; // Db_Shard0, Db_Shard1, ...

    public DbContext GetDbContextForUser(int userId)
    {
        int shardId = Math.Abs(userId.GetHashCode()) % _numberOfShards;
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseSqlServer(_connectionStrings[shardId])
            .Option
        return new AppDbContext(options);
    }
}
```

متى نستخدم Sharding ؟

| غير مناسب                                                       | مناسب                                                        |
|-----------------------------------------------------------------|--------------------------------------------------------------|
| قاعدة بيانات صغيرة (أقل من 100 جيجابايت)<br>التعقيد لا يستحق    | بيانات ضخمة جداً (مئات الجيجابايت أو أكثر)                   |
| استعلامات متعددة الأبعاد تتطلب تجميع بيانات عبر جميع المستخدمين | معدل كتابة مرتفع جداً آلاف المعاملات في الثاني               |
| فريق صغير (صعوبة إدارة العديد من قواعد البيانات)                | نموذج وصول طبيعي قابل للتجزئة<br>(مثل الوصول بمعرف المستخدم) |

## الجزء الرابع: مقارنة شاملة لجميع الحلول

| الحل               | صعوبة التطبيق | يعمل في بيئة موزعة؟ | أداء تحت ضغط 100 req/s | يحتاج بنية تحتية إضافية؟ |
|--------------------|---------------|---------------------|------------------------|--------------------------|
| Interlocked        | سهلة          | لا (خادم واحد)      | ممتاز                  | لا                       |
| Lock               | سهلة          | لا (خادم واحد)      | ممتاز                  | لا                       |
| ConcurrentQueue    | متوسطة        | لا (خادم واحد)      | ممتاز                  | لا                       |
| RowVersion + Retry | سهلة          | نعم                 | جيد                    | لا                       |
| SELECT FOR UPDATE  | متوسطة        | نعم                 | جيد                    | لا (MySQL 8.0.1+)        |
| RedLock            | صعبة          | نعم                 | جيد                    | نعم (3 × Redis)          |
| Sharding           | صعبة جداً     | نعم                 | ممتاز                  | نعم<br>خوادم DB متعددة   |

## الجزء الخامس: الحل الأنسب لمشروعنا مع الأسباب

### لماذا اخترنا RowVersion + Retry؟

بعد المقارنة الشاملة لجميع الحلول المتاحة، قررنا اعتماد التزامن المتفائل (Optimistic Concurrency) باستخدام RowVersion مع آلية إعادة المحاولة (Retry) كحل رئيسي لمشكلة التضارب في المخزون.

### الأسباب الهندسية للاختيار:

#### 1. يعمل في البيئة الموزعة التي يتطلبها المشروع:

المتطلب رقم 5 يطلب صراحة توزيع الأحمال (Load Distribution) الحلول التي تعتمد على الذاكرة فقط lock أو Interlocked و ConcurrentDictionary لا تعمل عندما يكون لدينا عدة خوادم. كل خادم له ذاكرته المستقلة، والأقفال المحلية لا تؤثر على الخوادم الأخرى.

RowVersion يعتمد على قاعدة البيانات المشتركة، وبالتالي يعمل بشكل طبيعي عبر جميع الخوادم. إذا كان لدينا 3 خوادم تعمل خلف Load Balancer ، فإن RowVersion يضمن عدم حدوث تضارب بينها.

## 2. لا يحتاج إلى بنية تحتية إضافية:

على عكس RedLock الذي يحتاج إلى 3 خوادم Redis مستقلة، وعلى عكس Sharding الذي يحتاج إلى عدة خوادم قاعدة بيانات، RowVersion يعمل مع قاعدة البيانات الموجودة فقط. هذا يقلل من التعقيد التشغيلي ويجعل المشروع قابلاً للتطبيق في بيئة تجريبية محلية.

## 3. يتكامل بسلاسة مع EF Core

مشروعنا يستخدم Entity Framework Core ORM أساسي RowVersion. هو جزء أصلي من EF Core ولا يحتاج إلى كتابة SQL خام أو إضافة مكتبات خارجية. كل ما نحتاجه هو إضافة الخاصية [Timestamp] وتكوين بسيط في DbContext

## 4. مناسب لحمولة تصل إلى 100 عملية متزامنة:

مع هذا المستوى من الضغط (100 طلب في الثانية على نفس المنتج)، نسبة التضارب ستكون صغيرة (أقل من 5%)، وإعادة المحاولة ستجح في معظم الحالات من المحاولة الثانية.

## 5. بسيط وقابل للفهم والتوثيق:

الكود الناتج سيكون بسيطاً وواضحاً.

لماذا لم نختار RedLock رغم أنه حل ممتاز للبيئة الموزعة؟

- يتطلب بنية تحتية إضافية: نحتاج إلى 3 خوادم Redis مستقلة على الأقل لتجنب Single Point of Failure
- تعقيد التشغيل: إدارة خوادم Redis الإضافية خارج نطاق المشروع التجريبي
- تكلفة زمنية إضافية: كل عملية قفل تتطلب اتصالات شبكة إلى عدة خوادم Redis
- غير ضروري لحمولة 100 عملية/ثانية RowVersion: كافٍ تماماً لهذا المستوى من الضغط

مع ذلك، إذا كنا سنطبق المشروع في بيئة إنتاجية حقيقية مع توزيع أحمال على 10 خوادم وحمولة تصل إلى 10,000 عملية/ثانية، لكان RedLock خياراً أفضل بكثير .

## الجزء السادس: التطبيق العملي

### 🚦 كيان المنتج Product Entity

```
public class Productpublic class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public int Stock { get; set; }

    [Timestamp] // هذا الحقل هو مفتاح حل التزامن المتفائل
    public byte[] RowVersion { get; set; } = Array.Empty<byte>();

    public DateTime LastUpdated { get; set; } = DateTime.UtcNow;
}
```

### 🚦 تكوين DbContext

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Property(p => p.RowVersion)
        .IsRowVersion(); // باستخدام الحقل لل التزامن المتفائل EF Core يخبر
}
```

### 🚦 خدمة الطلبات OrderService

```
public async Task<Order> CreateOrderDirectAsync(int userId, int productId, int
quantity, string cardNumber)
{
    if (quantity <= 0)
        throw new InvalidOperationException("Quantity must be positive.");
}
```



```

const int maxRetries = 3;
int attempt = 0;

while (attempt < maxRetries)
{
    // Start transaction on each attempt
    await using var transaction = await
_context.Database.BeginTransactionAsync();

    try
    {
        // Fetch product (done each attempt to get the latest RowVersion)
        var product = await _context.Products.FindAsync(productId);

        if (product == null)
            throw new InvalidOperationException("Product not found.");

        _logger.LogInformation($"Attempt {attempt + 1}: Current
stock={product.Stock}, requested={quantity},
RowVersion={Convert.ToBase64String(product.RowVersion ?? Array.Empty<byte>())}");

        if (product.Stock < quantity)
            throw new InvalidOperationException($"Insufficient stock. Available:
{product.Stock}");

        // Update stock
        product.Stock -= quantity;
        product.LastUpdated = DateTime.UtcNow;

        // May throw DbUpdateConcurrencyException if RowVersion has changed
        await _context.SaveChangesAsync();

        // Process payment (simulated)
        var paymentResult = await
_paymentGateway.ProcessPaymentAsync(product.Price * quantity, cardNumber);
        if (!paymentResult.Success)
            throw new InvalidOperationException("Payment failed.");

        // Create order
        var order = new Order
        {
            UserId = userId,
            OrderDate = DateTime.UtcNow,
            TotalAmount = product.Price * quantity,
            Status = OrderStatus.Paid,
            OrderItems = new List<OrderItem>
            {

```

```

        new OrderItem { ProductId = productId, Quantity = quantity,
UnitPrice = product.Price }
    }
};

await _context.Orders.AddAsync(order);

// Record payment transaction
var paymentTransaction = new PaymentTransaction
{
    Order = order,
    Amount = product.Price * quantity,
    GatewayReference = Guid.NewGuid().ToString(),
    Status = PaymentStatus.Success,
    PaymentDate = DateTime.UtcNow
};

await _context.PaymentTransactions.AddAsync(paymentTransaction);
await _context.SaveChangesAsync();

await transaction.CommitAsync();

_logger.LogInformation($" Attempt {attempt + 1} succeeded! Order
{order.Id}, remaining stock={product.Stock}");

return order;
}
catch (DbUpdateConcurrencyException ex)
{
    await transaction.RollbackAsync();
    attempt++;

    _logger.LogWarning(ex, $" Conflict on attempt {attempt}. Retrying...");

    if (attempt >= maxRetries)
        throw new InvalidOperationException("Concurrency conflict. Please
retry.");

    // Short delay before retrying
    await Task.Delay(50 * attempt);
}
catch
{
    await transaction.RollbackAsync();
    throw;
}
}

```

```
throw new InvalidOperationException("Concurrency conflict. Please retry.");
```

## Controller 🚦

```
[HttpPost("checkout")]
[HttpPost("checkout")]
public async Task<IActionResult> Checkout([FromBody] CheckoutRequest request)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    try
    {
        var userId = GetUserId(); // للاختبار: return 1;
        var order = await _orderService.CreateOrderDirectAsync(userId,
request.ProductId, request.Quantity, request.CardNumber);
        return Ok(new { orderId = order.Id, status = order.Status.ToString(), message
= "Order completed successfully" });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { error = ex.Message });
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Checkout error");
        return StatusCode(500, new { error = "An error occurred" });
    }
}
```

## الجزء السابع: طريقة الاختبار قبل وبعد الحل

### إعداد بيئة الاختبار

| العنصر                | القيمة                                      |
|-----------------------|---------------------------------------------|
| قاعدة البيانات        | MySQL محليا                                 |
| المنتج المستهدف       | Id = 1                                      |
| المخزون الابتدائي     | 5وحدات                                      |
| عدد الطلبات المتزامنة | 20 طلباً (كل طلب يشتري وحدة واحدة)          |
| أداة الاختبار         | مشروع Console مستقل (Race Condition Tester) |

### أداة الاختبار Race Condition Tester

```
using System.Net.Http.Json;

Console.WriteLine("Starting concurrency test (20 requests on a product with stock 5)");

var baseUrl = "https://localhost:7124";

// Ignore SSL certificate errors for local development
HttpClientHandler handler = new HttpClientHandler();
handler.ServerCertificateCustomValidationCallback = (sender, cert, chain, sslPolicyErrors) => true;
var client = new HttpClient(handler);

var tasks = new List<Task<HttpResponseMessage>>();

for (int i = 0; i < 20; i++)
{
    var requestBody = new {
        productId = 1,
        quantity = 1,
        cardNumber = $"4111111111111111{i}" // different card number per request
    };
    tasks.Add(client.PostAsJsonAsync($"{baseUrl}/api/orders/checkout", requestBody));
}

Console.WriteLine("Sending 20 concurrent requests...");
```

```

var responses = await Task.WhenAll(tasks);

int successCount = responses.Count(r => r.IsSuccessStatusCode);
int failCount = responses.Count(r => !r.IsSuccessStatusCode);

Console.WriteLine($"Successful requests: {successCount}");
Console.WriteLine($"Failed requests: {failCount}");
Console.WriteLine($"Expected: 5 success / 15 failure");

foreach (var resp in responses.Where(r => !r.IsSuccessStatusCode).Take(3))
{
    var errorText = await resp.Content.ReadAsStringAsync();
    Console.WriteLine($"Error: {errorText}");
}

```

نتائج الاختبار قبل تطبيق الحل (بدون حماية)

```

C:\Users\meryem\source\repos\ECommerceBack\LoadTester>dotnet run
Starting concurrency test (20 requests on a product with stock 5)
Sending 20 concurrent requests...
Successful requests: 20
Failed requests: 0

```

الشكل (1)

نتائج الاختبار بعد تطبيق الحل (RowVersion + Retry)

| Max Rows: 1000 |                   |            |       |  | موجه الاوامر                                                                                                                                                                                                                                                                                                                    |  |
|----------------|-------------------|------------|-------|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Id             | Name              | Price      | Stock |  |                                                                                                                                                                                                                                                                                                                                 |  |
| 1              | Laptop Ultrabook  | 5000.00    | 0     |  | Starting concurrency test (20 requests on a product with stock 5)<br>Sending 20 concurrent requests...<br>Successful requests: 5<br>Failed requests: 15<br>Error: {"error": "Insufficient stock. Available: 0"}<br>Error: {"error": "Insufficient stock. Available: 0"}<br>Error: {"error": "Insufficient stock. Available: 0"} |  |
| 2              | New Tablet        | 750.00     | 22    |  |                                                                                                                                                                                                                                                                                                                                 |  |
| 3              | Mechanical Key... | 250.00     | 30    |  |                                                                                                                                                                                                                                                                                                                                 |  |
| 4              | USB-C Hub         | 150.00     | 25    |  |                                                                                                                                                                                                                                                                                                                                 |  |
| 5              | 4K Monitor        | 1200.00    | 8     |  |                                                                                                                                                                                                                                                                                                                                 |  |
| 8              | Laptop Ultrabook  | 2333303.00 | 2330  |  |                                                                                                                                                                                                                                                                                                                                 |  |
| 9              | Laptop Ultrabook  | 2333303.00 | 2330  |  |                                                                                                                                                                                                                                                                                                                                 |  |
| 9              | NULL              | NULL       | NULL  |  |                                                                                                                                                                                                                                                                                                                                 |  |

الشكل (2)

جدول مقارنة النتائج

| المعيار                               | قبل الحل       | بعد الحل  |
|---------------------------------------|----------------|-----------|
| عدد الطلبات الناجحة                   | (غير منطقي) 16 | (منطقي) 5 |
| عدد الطلبات الفاشلة                   | 4              | 15        |
| المخزون النهائي                       | (كارثة) -11    | (صحيح) 0  |
| (Over-selling) وجود بيع زائد          | نعم            | لا        |
| Lost Update وجود                      | نعم            | لا        |
| في السجلات Concurrency وجود استثناءات | لا             | نعم       |

## المتطلب غير الوظيفي الثاني: إدارة الموارد الحاسوبية

### Resource Management & Capacity Control

#### الجزء الأول: شرح عام عن المشكلة

ما هي إدارة الموارد الحاسوبية؟

في أي نظام برمجي، الموارد الحاسوبية مثل المعالج CPU و الذاكرة RAM و اتصالات قاعدة البيانات و الخيوط Threads هي موارد محدودة.

عندما يتعامل النظام مع آلاف الطلبات المتزامنة، يمكن لهذه الموارد أن تستنفد بسرعة، مما يؤدي إلى:

- تباطؤ شديد في الاستجابة (زيادة زمن الاستجابة)
- انهيار النظام بسبب استهلاك كل الذاكرة أو وصول عدد الخيوط إلى الحد الأقصى
- فشل الخوادم (Cascading Failure) حيث يؤدي فشل جزء إلى انهيار الأجزاء الأخرى

الهدف من هذا المتطلب هو: التحكم في عدد العمليات المتوازية التي ينفذها النظام، بحيث لا يستهلك الموارد بشكل مفرط (فيؤدي إلى الانهيار) ولا يقللها بشكل يبطئ الاستجابة بشكل غير مقبول

في سياق مشروعنا، نحتاج إلى وضع حدود للموارد التي يمكن لخادم التطبيق استخدامها لمعالجة الطلبات

#### المصطلحات الأساسية

| المصطلح    | المعنى                                           |
|------------|--------------------------------------------------|
| Throughput | عدد الطلبات التي يمكن للنظام معالجتها في الثانية |
| Latency    | الوقت المستغرق لمعالجة طلب واحد                  |

|                                                                     |                     |
|---------------------------------------------------------------------|---------------------|
| استنفاد مورد حاسوبي (مثل الذاكرة، الخيوط) مما يؤدي إلى فشل النظام   | Resource Exhaustion |
| تحديد عدد الطلبات المسموح بها من عميل أو مصدر معين خلال فترة زمنية  | Rate Limiting       |
| الحد الأقصى لعدد الطلبات أو الخيوط التي تعالج بالتوازي              | Concurrency Limit   |
| آلية تمنع النظام من استقبال طلبات أكثر مما يستطيع معالجته           | Backpressure        |
| مجموعة من الخيوط الجاهزة للتنفيذ، يديرها نظام التشغيل أو إطار العمل | Thread Pool         |

## الجزء الثاني: شرح المشكلة في المشروع

السيناريو: هجوم 1000 طلب مفاجئ

لنفترض أن نظام التجارة الإلكترونية يعمل بشكل طبيعي مع 50 مستخدمًا متزامنًا. فجأة، بسبب حملة تسويقية أو تخفيضات، يرتفع العدد إلى 1000 طلب في الثانية

في نظام غير محمي بالموارد، سيحدث التالي:

- خادم الويب Kestrel في ASP.NET Core سيحاول إنشاء خيوط جديدة Threads لمعالجة كل طلب.
- عدد الخيوط سيتضخم بسرعة إلى 1000 أو أكثر.
- كل خيط يستهلك ذاكرة حوالي 1 ميجابايت لكل خيط في 1000 NET. خيط = 1 جيجابايت ذاكرة إضافية.
- ال Thread Pool في NET. محدود الحجم افتراضيًا حوالي 32767 خيطًا، لكن إنشاء خيوط كثيرة جدًا يسبب:
  - تغير السياق المتكرر Context Switching بين الخيوط، مما يبطئ المعالج.
  - تبادل كبير للذاكرة
  - قد يصل النظام إلى نقطة الانهيار حيث يتوقف عن الاستجابة
- قاعدة البيانات ستتلقى 1000 استعلام متزامن، وقد تستهلك اتصالاتها Connection Pool وتصل إلى الحد الأقصى (غالبًا 100 اتصال افتراضيًا). الطلبات الإضافية ستنظر أو تفشل.

**النتيجة:** بدلاً من أن يخدم النظام 1000 طلب ببطء وهو أمر مقبول في بعض الحالات، فإنه قد ينهار تمامًا ويصبح غير

متاح للجميع. هذا ما يُعرف بـ انهيار الدومينو **Cascading Failure**



## المشكلة في مشروعنا

في مشروعنا، لدينا عدة عمليات حساسة من حيث استهلاك الموارد:

| العملية                                    | استهلاك الموارد                                    | الخطر                                                                |
|--------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------|
| <b>OrderService.CreateOrderDirectAsync</b> | يقرأ ويكتب في قاعدة البيانات،<br>ويقوم بمحاكاة دفع | عالي - يمكن أن يستهلك العديد<br>من الخيوط المنتظرة                   |
| <b>ProductsController.GetAll</b>           | يقرأ كل المنتجات من قاعدة<br>البيانات              | متوسط - قد يكون ثقيلاً على<br>الذاكرة إذا كان عدد المنتجات<br>كبيراً |
| <b>ReportsController.GetDailySummary</b>   | يجمع بيانات من جداول كبيرة<br>(Orders, OrderItems) | مرتفع - قد يستغرق وقتاً طويلاً<br>ويستهلك معالجاً                    |

بدون تحكم في الموارد، طلب واحد بطيء مثل تقرير مبيعات قد يستهلك خيطاً لمدة 10 ثوانٍ، وإذا تكرر 100 مرة، ستشغل 100 خيط وتستهلك موارد النظام، مما يؤثر على سرعة الطلبات الأخرى.

## الجزء الثالث: الحلول الممكنة

هناك عدة طرق في .NET. للتحكم في استهلاك الموارد وحماية النظام من الانهيار تحت الضغط:

### الحل الأول: تقييد عدد الخيوط المتزامنة باستخدام SemaphoreSlim

**الفكرة:** استخدام SemaphoreSlim للسماح بعدد محدد من الخيوط لدخول منطقة حرجية مثل عملية الدفع.

أي خيط إضافي ينتظر دوره

```
private static readonly SemaphoreSlim _paymentSemaphore = new SemaphoreSlim(10); //
```

طلبات دفع متزامنة كحد أقصى 10

```
public async Task<Order> CreateOrderAsync(...)
{
    await _paymentSemaphore.WaitAsync();
    try
    {
        // عملية الدفع والمخزون
    }
    finally
    {
        _paymentSemaphore.Release();
    }
}
```

#### المميزات:

- بسيط وفعال.
- يسمح بالتحكم الدقيق في عدد العمليات المتزامنة لوظيفة معينة.

#### العيوب :

- لا يمنع وصول عدد كبير من الطلبات إلى النظام لا يعمل ك Rate Limiting على مستوى المدخل
- يجب توزيع Semaphore بشكل صحيح عبر الخدمات المختلفة.

### الحل الثاني: تحديد حجم Thread Pool

الفكرة: تغيير الإعدادات الافتراضية لـ ThreadPool في .NET.

```
ThreadPool.SetMinThreads(100, 100);
ThreadPool.SetMaxThreads(500, 500);
```

### المميزات:

- يؤثر على كل عمليات التطبيق

### العيوب:

- ضبط هذه القيم يحتاج إلى خبرة ومعرفة دقيقة بالحمل.
- يمكن أن يؤدي إلى تجويع الخيوط (Thread Starvation) إذا كانت القيم منخفضة جداً.
- لا يفرق بين أنواع الطلبات (مهمة مقابل غير مهمة).

## الحل الثالث: Rate Limiting

**الفكرة:** استخدام الوسيط Middleware المدمج في ASP.NET Core لتحديد عدد الطلبات المسموح بها لكل عميل IP أو مستخدم أو لكل مسار Endpoint خلال فترة زمنية معينة.

```
builder.Services.AddRateLimiter(options =>
{
    options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(
        httpContext => RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.User.Identity?.Name ??
httpContext.Request.Headers.Host.ToString(),
            factory: partition => new FixedWindowRateLimiterOptions
            {
                AutoReplenishment = true,
                PermitLimit = 100,
                QueueLimit = 0,
                Window = TimeSpan.FromSeconds(60)
            }
        ));
});

app.UseRateLimiter();
```

### المميزات :

- حل متكامل في ASP.NET Core
- يدعم استراتيجيات متعددة (Fixed Window, Sliding Window, Token Bucket, Concurrency)
- يمكن تطبيقه على مستوى العميل أو المسار.

## العيوب:

- لا يتحكم في استهلاك الموارد داخل العملية (الخيوط، الذاكرة) بنفس فعالية SemaphoreSlim
- قد يرفض طلبات بشكل غير عادل إذا تم تكوينه بشكل سيء.

## الحل الرابع: استخدام Message Queue

**الفكرة:** بدلاً من معالجة الطلبات فوراً، نضعها في قائمة انتظار مثل ConcurrentQueue أو RabbitMQ ونعالجها بعدد محدد من العمال Workers هذا يضمن ألا يتجاوز عدد الطلبات قيد المعالجة سقفًا معيناً.

```
// فقط ضع الطلب في القائمة API واجهة
_queue.Enqueue(request);
return Accepted(); // 202

// BackgroundService خلفية في
while (await _channel.Reader.WaitToReadAsync())
{
    var request = await _channel.Reader.ReadAsync();
    await ProcessOrderAsync(request);
}
```

## المميزات :

- فصل كامل بين استلام الطلبات ومعالجتها.
- يمكن تحديد عدد العمال (workers) بسهولة.
- يحمي النظام من الانفجار المفاجئ.

## العيوب :

- لا يعطي رداً فورياً للمستخدم يحتاج إلى إشعار لاحق.
- تعقيد إضافي.

## الحل الخامس: تقييد عدد الاتصالات المتزامنة لقاعدة البيانات

الفكرة : ضبط حجم Connection Pool في سلسلة الاتصال:

```
"ConnectionStrings": {"ConnectionStrings": {
    "DefaultConnection": "Server=...;Max Pool Size=50;"
}}
```

المميزات: يمنع استنفاد اتصالات قاعدة البيانات.

العيوب: لا يتحكم في موارد الخادم الأخرى.

## الجزء الرابع: مقارنة الحلول

| الحل                    | صعوبة التطبيق | يمنع انهيار الخادم؟         | يحد من عدد الطلبات الواردة؟ | يتحكم في الخيوط؟ | مناسب لحمولة 1000 طلب/ثانية؟ |
|-------------------------|---------------|-----------------------------|-----------------------------|------------------|------------------------------|
| SemaphoreSlim           | سهلة          | جزئياً (على مستوى الوظيفة)  | لا                          | نعم              | نعم (مع تكوين جيد)           |
| ضبط ThreadPool          | صعبة          | نعم                         | لا                          | نعم              | لا (غير دقيق)                |
| Rate Limiting (ASP.NET) | متوسطة        | نعم (منع الوصول)            | نعم                         | لا               | نعم                          |
| Message Queue           | عالية         | نعم                         | نعم (عبر العمال)            | نعم              | ممتاز                        |
| Connection Pool         | سهلة          | جزئياً (قاعدة البيانات فقط) | لا                          | لا               | محدود                        |

## الجزء الخامس: الحل الأنسب لمشروعنا مع الأسباب

### الحل المختار

بناءً على تحليل طبيعة النظام وحجم الحمل المتوقع حتى 1000 طلب في الثانية على أقصى تقدير، تم اعتماد آلية قفل على مستوى الذاكرة باستخدام SemaphoreSlim يتم تطبيقها عبر خدمة مركزية IConcurrencyManager يتم تحديد حد أقصى لعدد الطلبات المتزامنة التي تدخل إلى دالة إنشاء الطلب CreateOrderDirectAsync لكل منتج على حدة، وفي حالتنا تم اختيار الحد الأقصى 10 عمليات متزامنة

### لماذا هذا الحل؟

- التحكم الدقيق في استهلاك الموارد الحاسوبية:
  - عملية إنشاء الطلب هي الأكثر استهلاكاً للوقت في النظام: فهي تشمل قراءة وكتابة قاعدة البيانات، التعامل مع `RowVersion` ( وإعادة المحاولات)، ومحاكاة بوابة الدفع. إذا سمحنا بأعداد كبيرة من الطلبات المتزامنة غير المقيدة، فإن `ThreadPool` في .NET. سيتضخم بشكل كبير، مما يؤدي إلى:
    - ارتفاع غير خطي في زمن الاستجابة.
    - استنزاف الذاكرة بسبب كثرة الكائنات المؤقتة.
    - كثافة عالية في عمل Garbage Collector ظهور Gen0 و Gen1 بشكل متكرر.
- باستخدام SemaphoreSlim ، نضمن أن عدداً محدوداً فقط من الخيوط يمكنه تنفيذ العمليات الثقيلة في أي لحظة، مما يحافظ على استقرار النظام تحت الضغط.
- خفة الوزن وعدم الحاجة إلى بنية تحتية إضافية
  - على عكس الحلول التي تتطلب إعداد خوادم خارجية مثل `Redis Distributed Lock` أو `RabbitMQ` ، يعمل هذا الحل بالكامل في الذاكرة ولا يضيف أي تكاليف تشغيلية أو تعقيدات في النشر. هذا يجعله مناسباً تماماً لمرحلة التطوير والتجريب، كما أنه يظل قابلاً للتطوير لاحقاً إلى حل موزع مثل `RedLock` إذا توسع النظام.
- التكامل السلس مع التزامن المتقابل

الحل لا يتعارض مع آلية Optimistic Concurrency المطبقة بالفعل في دالة إنشاء الطلب. فبينما يحد SemaphoreSlim من عدد العمليات المتزامنة على مستوى التطبيق، تقوم RowVersion بحماية البيانات على مستوى قاعدة البيانات. هذا المزج يعطي طبقتين من الأمان:

- طبقة التطبيق: تمنع الاندفاع الكبير للطلبات.
- طبقة قاعدة البيانات: تتعامل مع حالات التضارب النادرة التي قد تحدث حتى مع وجود القفل.

### لماذا لم نستخدم Rate Limiting المدمج في ASP.NET Core

Rate Limiting يحد من عدد الطلبات الواردة لكل عميل IP أو مستخدم خلال نافذة زمنية. لكن المشكلة الأساسية ليست في عدد الطلبات الواردة، بل في عدد الخيوط التي تدخل إلى المنطقة الحرجة في نفس الوقت. حتى لو طبقنا حداً لكل عميل، فإن عشرات العملاء المختلفين قد يرسلون طلبات متزامنة ويستنزفون الخيوط معاً. لذلك، كان التحكم في التزامن الداخلي أكثر فعالية من تقييد المعدل عند المدخل.

### لماذا لم نستخدم Message Queue:

- الرد الفوري: عملية الشراء تتطلب رداً فورياً للمستخدم. استخدام قوائم الانتظار يحول النظام إلى نمط غير متزامن، مما يؤخر الرد أو يتطلب آليات إضافية للإشعار.
- التعقيد غير المبرر: إدارة وسائط رسائل مثل RabbitMQ يزيد من تعقيد التشغيل والصيانة، ولا يوجد مبرر قوي له في ظل الحمل المتوقع (حتى 1000 طلب/ثانية) بعد تطبيق التحكم بـ SemaphoreSlim

## التنفيذ العملي في المشروع

### خدمة IConcurrencyManager وتطبيقها

أنشأنا واجهة IConcurrencyManager تحتوي على دالة ExecuteWithLimitAsync التي تقبل مفتاح المورد resourceKey، والدالة المراد تنفيذها action، والحد الأقصى لعدد العمليات المتزامنة maxConcurrency

```
public interface IConcurrencyManager
{
    Task<T> ExecuteWithLimitAsync<T>(string resourceKey, Func<Task<T>> action, int
maxConcurrency = 10);
}
```

ثم قمنا بتنفيذها باستخدام ConcurrentDictionary<string, SemaphoreSlim> لضمان وجود قفل مستقل لكل مفتاح (مثل كل منتج):

```
public class ConcurrencyManager : IConcurrencyManager
{
    private static readonly ConcurrentDictionary<string, SemaphoreSlim> _semaphores =
new();

    public async Task<T> ExecuteWithLimitAsync<T>(string resourceKey, Func<Task<T>>
action, int maxConcurrency = 10)
    {
        var semaphore = _semaphores.GetOrAdd(resourceKey, _ => new
SemaphoreSlim(maxConcurrency, maxConcurrency));
        await semaphore.WaitAsync();
        try
        {
            return await action();
        }
        finally
        {
            semaphore.Release();
        }
    }
}
```

### تسجيل الخدمة في Program.cs

تم تسجيل Singleton كـ IConcurrencyManager لضمان مشاركة نفس الأقفال عبر جميع الطلبات:

```
builder.Services.AddSingleton<IConcurrencyManager, ConcurrencyManager>();
```

### تطبيق الحد على OrderService

في دالة CreateOrderDirectAsync الموجودة في OrderService ، قمنا بتغليف منطق الطلب بالكامل داخل

ExecuteWithLimitAsync مع تحديد maxConcurrency = 10

```
public async Task<Order> CreateOrderDirectAsync(int userId, int productId, int
quantity, string cardNumber)
```



```

{
    return await _concurrencyManager.ExecuteWithLimitAsync(
        $"checkout_{productId}",
        async () =>
        {
            // معاملة قاعدة البيانات، محاكاة الدفع، RowVersion مع منطق إنشاء الطلب الأصلي
            // ...
            return order;
        },
        maxConcurrency: 10);
}

```

## منهجية الاختبار باستخدام BenchmarkDotNet

### أداة الاختبار

استخدمنا مكتبة BenchmarkDotNet المعيار في .NET. لقياس الأداء لأنها توفر:

- قياسات دقيقة للوقت
- قياس الذاكرة المخصصة ('Allocated') وعمليات ('Gen0', 'Gen1') و GC عبر '[MemoryDiagnoser]'
- إمكانية التحكم في عدد التكرارات ('IterationCount=5') والتسخين ('WarmupCount=2') للحصول على نتائج مستقرة.

### إعداد الاختبار

- النسخة NoLimit و قبل الحل OrderService: عادي بدون استخدام ConcurrencyManager يسمح بعدد غير محدود من الطلبات المتزامنة.
- النسخة WithLimit بعد الحل OrderService : يغلف منطق الطلب بـ ConcurrencyManager والحد = 10.
- السيناريوهات: تم اختبار أحمال مختلفة من الطلبات المتزامنة (ConcurrentRequests = 5, 10, 30) يتم إرسال جميع الطلبات باستخدام Task.WhenAll محاكياً التزامن الحقيقي.

## الكود المستخدم في BenchmarkDotNet

تم تعريف كلاس OrderConcurrencyBenchmark يحتوي على:

خاصية `[Params(5, 10, 30)] public int ConcurrentRequests { get; set; }`

`[Benchmark(Baseline = true)]` -لدالة `CreateMultipleOrders_NoLimit`

`[Benchmark]` -لدالة `CreateMultipleOrders_WithLimit`

`[MemoryDiagnoser(true)]` -لتفعيل قياس الذاكرة.

كل بنشمارك يقوم بـ `Task.WhenAll(Enumerable.Range(0, ConcurrentRequests).Select(...))` حيث يتم استدعاء `CreateOrderAsync` لكل طلب.

## النتائج والتحليل

| Method                         | ConcurrentRequests | Mean       | Error      | StdDev      | Ratio | RatioSD | Gen0      | Gen1      | Allocated | Alloc Ratio |
|--------------------------------|--------------------|------------|------------|-------------|-------|---------|-----------|-----------|-----------|-------------|
| CreateMultipleOrders_NoLimit   | 5                  | 305.6 ms   | 567.2 ms   | 87.78 ms    | 1.09  | 0.49    | -         | -         | 2.11 MB   | 1.00        |
| CreateMultipleOrders_WithLimit | 5                  | 474.0 ms   | 329.0 ms   | 85.43 ms    | 1.70  | 0.68    | -         | -         | 2.34 MB   | 1.11        |
| CreateMultipleOrders_NoLimit   | 10                 | 1,675.7 ms | 1,825.6 ms | 474.10 ms   | 1.06  | 0.38    | 1000.0000 | -         | 8.03 MB   | 1.00        |
| CreateMultipleOrders_WithLimit | 10                 | 1,820.8 ms | 1,437.9 ms | 373.43 ms   | 1.15  | 0.36    | 1000.0000 | -         | 8.04 MB   | 1.00        |
| CreateMultipleOrders_NoLimit   | 30                 | 6,859.7 ms | 6,584.0 ms | 1,018.88 ms | 1.02  | 0.20    | 5000.0000 | 3000.0000 | 34 MB     | 1.00        |
| CreateMultipleOrders_WithLimit | 30                 | 2,301.7 ms | 888.9 ms   | 230.85 ms   | 0.34  | 0.06    | 2000.0000 | 1000.0000 | 13.55 MB  | 0.40        |

الشكل (3)

## التحليل الهندسي:

🚦 عند 5 و 10 طلبات: الأداء متقارب، مع زيادة طفيفة في الزمن والذاكرة للنسخة المقيدة بسبب آلية الانتظار `WaitAsync` هذا الانخفاض الطفيف (حوالي 150 مللي ثانية) يعتبر مقبولاً جداً مقارنة بالمكاسب الكبيرة تحت الضغط العالي.

🚦 عند 30 طلباً (الضغط العالي): النسخة `NoLimit` انهارت أدائياً زمن الاستجابة تضاعف 3 مرات مقارنة بـ 10 طلبات، والذاكرة تضاعفت 4 مرات، وظهرت آلاف عمليات `GC` بينما النسخة `WithLimit` حافظت على استقرار نسبي زاد الزمن بشكل طبيعي (من 1.8 إلى 2.3 ثانية)، وزادت الذاكرة بشكل محدود، وانخفض ضغط `GC` بشكل كبير.

✚ **تحسن كمي:** عند 30 طلباً، حققنا تخفيضاً في زمن الاستجابة بنسبة ~66% (من 6.86 إلى 2.30 ثانية)، وتخفيضاً في الذاكرة المخصصة بنسبة ~60% (من 34 إلى 13.55 ميجابايت)، وتخفيضاً في عمليات Gen0 بنسبة 60% (من 5000 إلى 2000) و Gen1 بنسبة 67% (من 3000 إلى 1000).

**الاستنتاج:** استخدام ConcurrencyManager يحمي النظام من الانهيار تحت الضغط العالي، ويحافظ على استقرار استهلاك الموارد، مع تضحية طفيفة في الأداء للأحمال المنخفضة (وهي مقبولة هندسياً).

## المتطلب غير الوظيفي الثالث:

### المعالجة غير المتزامنة Asynchronous Queues

#### الجزء الأول: شرح عام عن المفهوم

ما هي المعالجة غير المتزامنة؟

في أنظمة البرمجيات التقليدية، عندما يرسل المستخدم طلباً (مثل عملية شراء)، يقوم الخادم بتنفيذ جميع الخطوات المطلوبة بشكل متسلسل ومباشر قبل إعادة الرد إلى المستخدم. هذا يعني أن المستخدم ينتظر حتى اكتمال كل شيء: تحديث قاعدة البيانات، معالجة الدفع، إنشاء الفاتورة، إرسال البريد الإلكتروني، تسجيل الإحصائيات، إلخ.

في هذا النموذج المتزامن **Synchronous** ، كلما زادت المهام التي يجب تنفيذها في الطلب الواحد، كلما زاد وقت انتظار المستخدم.

هذا يؤدي إلى:

- تجربة مستخدم سيئة : المستخدم ينتظر ثوانٍ طويلة (أو حتى دقائق) قبل رؤية رسالة التأكيد.
- استهلاك موارد الخادم : كل خيط من خيوط المعالجة يبقى مشغولاً طوال مدة الطلب، مما يقلل من قدرة النظام على معالجة طلبات أخرى.
- زيادة احتمالية الفشل : كلما طالت مدة الطلب، زادت احتمالية حدوث أخطاء (مثل انقطاع الشبكة، انتهاء المهلة).

المعالجة غير المتزامنة **Asynchronous Processing** تعني فصل المهام التي لا يحتاج المستخدم إلى انتظار نتائجها عن مسار الطلب الرئيسي.

ببساطة:

- يستقبل الخادم الطلب.

- يقوم بالحد الأدنى من الخطوات الضرورية (مثل التحقق من صحة الطلب، حفظ البيانات الأساسية).
- يعيد رداً للمستخدم فوراً (على سبيل المثال: "تم استلام طلبك وسيتم معالجته").
- يتم نقل المهام الثقيلة أو غير العاجلة (مثل إصدار الفاتورة، إرسال الإشعارات، تحديث التقارير) إلى قائمة انتظار

Queue لمعالجتها لاحقاً بواسطة عمليات منفصلة تعمل في الخلفية Background Workers

في سياق مشروعنا ،المستخدم مهتم في المقام الأول بتنفيذ عملية الشراء وتأكيد الدفع. أما إنشاء الفاتورة بصيغة PDF ، أو إرسال بريد إلكتروني للتأكيد، أو تحديث إحصائيات المبيعات، فهي مهام يمكن تأجيلها دون التأثير على جوهر العملية. هذا هو جوهر المتطلب الثالث.

## المصطلحات الأساسية

| المصطلح                        | المعنى                                                                                                            |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Synchronous Processing</b>  | تنفيذ المهام بشكل متسلسل داخل نفس الطلب، ويبقى المستخدم في انتظار النتيجة.                                        |
| <b>Asynchronous Processing</b> | فصل المهام إلى مسارين: مسار فوري ومسار خلفي غير متزامن.                                                           |
| <b>Message Queue</b>           | بنية بيانات (أو نظام وساطة) تخزن الرسائل (الطلبات) مؤقتاً حتى يتم استهلاكها بواسطة المعالجات.                     |
| <b>Producer</b>                | الكيان الذي يضيف الرسائل إلى قائمة الانتظار عادةً واجهة API                                                       |
| <b>Consumer</b>                | الكيان الذي يسحب الرسائل من قائمة الانتظار ويعالجها .                                                             |
| <b>Background Service</b>      | خدمة تعمل في الخلفية ضمن تطبيق NET Core . وتنفذ مهام دورية أو مستمرة                                              |
| <b>Fire-and-Forget</b>         | نمط إرسال الرسالة حيث لا ينتظر المرسل أي رد أو تأكيد من المستهلك.                                                 |
| <b>At-least-once delivery</b>  | ضمان أن الرسالة ستعالج مرة واحدة على الأقل (قد تتكرر، لكنها لن تضيع).                                             |
| <b>Idempotency</b>             | خاصية العملية التي تجعل تنفيذها عدة مرات يعطي نفس النتيجة مثل تنفيذها مرة واحدة (مهم عند معالجة الرسائل المكررة). |

## الجزء الثاني: شرح المشكلة في المشروع

### السيناريو الحالي المعالجة المتزامنة

دعنا نراجع تدفق عملية الشراء في مشروعنا كما هو مطبق حالياً في دالة `CreateOrderDirectAsync` :

- بدء المعاملة `Transaction` مع قاعدة البيانات.
- جلب المنتج من قاعدة البيانات والتحقق من المخزون.
- تحديث المخزون خفض الكمية وحفظ التغيير.
- معالجة الدفع .
- إنشاء كائن الطلب وحفظه في قاعدة البيانات.
- إنشاء سجل معاملة الدفع وحفظه.
- إنهاء المعاملة.
- إعادة الرد للمستخدم .

لاحظ أنه لا توجد أي خطوة غير ضرورية هنا؟ في الواقع، هناك العديد من المهام التي يمكن تأجيلها أو تنفيذها بشكل غير متزامن:

- إصدار الفاتورة `Invoice`: يحتاج الطلب فقط إلى سجل فاتورة في قاعدة البيانات، ولكن إنشاء ملف `PDF` وتخزينه يمكن أن يتم لاحقاً.
- إرسال البريد الإلكتروني للمستخدم : ليس ضرورياً لإتمام الصفقة.
- تحديث إحصائيات المبيعات (مثل إجمالي المبيعات اليومية، عدد الطلبات): يمكن تجميعها لاحقاً عبر وظيفة دفعات (Batch Processing)
- تسجيل الأحداث (Event Logging) للمراقبة والتحليلات: يمكن تخزينها في قائمة انتظار ثم كتابتها بشكل دوري.
- إشعار المسؤول (مثل عملية شراء كبيرة): غير عاجل.

### المشكلة: لماذا هذا التصميم الحالي غير مناسب؟

- زمن استجابة مرتفع : المستخدم ينتظر حتى يتم إنشاء الفاتورة (حتى لو كان ذلك يستغرق 0.5 ثانية إضافية). في الواقع، قد يستغرق إرسال البريد الإلكتروني عدة ثوانٍ بسبب تأخر الشبكة.

- هدر موارد الخادم : الخيط الذي يعالج طلب المستخدم يظل مشغولاً بإنشاء الفاتورة وإرسال البريد الإلكتروني، بينما كان يمكن تحريره للتعامل مع طلبات أخرى.
- تأثير الدومينو : إذا ازدحم النظام، فإن هذه المهام الإضافية تزيد الحمل على الخادم وقاعدة البيانات، مما قد يسبب انهياراً. صعوبة إعادة المحاولة : إذا فشل إرسال البريد الإلكتروني بسبب مشكلة مؤقتة، فقد يفقد النظام القدرة على إعادة المحاولة .

### المتطلب المطلوب:

#### المعالجة غير المتزامنة Asynchronous Queues

نقل المهام التي لا يحتاج المستخدم لانتظارها مثل إصدار الفواتير أو إرسال الإشعارات خارج المسار الرئيسي للطلب. هذا يعني أن المستخدم لا يجب أن ينتظر اكتمال هذه المهام. يجب أن يحصل على رد فوري بأن طلبه قد تم استلامه ومعالجته أو رفضه بسبب نقص المخزون، ويتم تفويض المهام الثانوية إلى قائمة انتظار.

### الجزء الثالث: الحلول الممكنة

هناك عدة طرق لتنفيذ المعالجة غير المتزامنة في .NET ، تتراوح في التعقيد والقدرات.

#### الحل الأول: استخدام Fire-and-Forget مع Task.Run

##### الفكرة الأساسية

في هذا النمط، يقوم الخادم API بعد إتمام الخطوات الأساسية للطلب (مثل تحديث المخزون وإنشاء الطلب) بإطلاق مهمة خلفية Background Task باستخدام Task.Run، ثم يعيد الرد إلى المستخدم دون انتظار اكتمال هذه المهمة . المهمة الخلفية تُنفذ بشكل متزامن مع الاستجابة، ولكنها لا تحجبها.

```
// بعد نجاح الطلب
_ = Task.Run(() => SendOrderConfirmationEmailAsync(order));
_ = Task.Run(() => GenerateInvoicePdfAsync(order));
return Ok(order); // يرد فوراً دون انتظار البريد أو الفاتورة
```

## آلية العمل

Task.Run تقوم بجدولة العمل على خيط من مجمّع الخيوط ThreadPool

الرمز = \_ (تجاهل المهمة) يتجاهل كائن Task، مما يعني أن الكود لن ينتظر اكتمال المهمة ولن يلتقط أي استثناءات قد تحدث داخلها.

الخيط الذي ينفذ المهمة الخلفية يكون مستقلاً عن خيط معالجة الطلب الأصلي.

## مميزات هذا الحل

| الميزة                 | الشرح                                                             |
|------------------------|-------------------------------------------------------------------|
| بساطة التنفيذ          | لا حاجة لمكتبات إضافية، ولا لقواعد بيانات، ولا لإعدادات معقدة.    |
| سرعة الرد على المستخدم | يتحرر خيط الاستجابة فوراً، مما يقلل زمن الاستجابة بشكل ملحوظ.     |
| لا يستهلك موارد إضافية | يستخدم ThreadPool الموجود أصلاً في <a href="#">ASP.NET Core</a> . |

## عيوب خطيرة (لماذا لا نستخدمه في الإنتاج؟)

| العيوب                                        | الشرح                                                                                                              |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| فقدان المهمة Loss of Task                     | إذا انتهت مهلة الطلب أو تعطل التطبيق قبل اكتمال المهمة الخلفية، تختفي المهمة نهائياً لا توجد استمرارية Persistence |
| لا إعادة محاولة تلقائية                       | إذا فشلت المهمة مثلاً: انقطاع الشبكة عند إرسال البريد، لا توجد آلية لإعادة تشغيلها.                                |
| لا تحكم في التزامن                            | إطلاق آلاف المهام الخلفية قد يستنزف ThreadPool ويؤدي إلى تدهور أداء التطبيق ككل.                                   |
| صعوبة المراقبة والتتبع                        | لا توجد واجهة لعرض المهام الناجحة أو الفاشلة. لا يمكن معرفة سبب الفشل إلا من خلال السجلات Logs إذا تم تسجيلها.     |
| استثناءات غير معالجة<br>Unobserved exceptions | إذا أثارت المهمة استثناءً ولم يتم التقاطه، فإنه قد يمر دون أن يلاحظه أحد أو قد يتسبب في إنهاء العملية.             |



لا يعمل في البيئات الموزعة إذا كان لديك عدة خوادم Load Balancing ، فإن المهمة تُنفذ فقط على الخادم الذي استلم الطلب. لا يوجد تنسيق بين الخوادم.

أين يمكن (أو لا يمكن) استخدام هذا الأسلوب؟

| السيناريو                                            | القرار        | التبرير                                                         |
|------------------------------------------------------|---------------|-----------------------------------------------------------------|
| تطوير نموذج أولي Prototype                           | مقبول مؤقتاً  | لتوضيح الفكرة فقط، مع التوثيق الواضح أنه غير آمن للإنتاج.       |
| تسجيل إحصائيات غير حرجية (مثل عدد الزوار)            | مقبول         | لأن فقدان بعض الإحصائيات ليس كارثة.                             |
| إرسال إشعارات داخلية غير عاجلة مع إعادة محاولة يدوية | غير مرغوب فيه | احتمال فقدان مرتفع.                                             |
| أي عملية تتطلب موثوقية (إصدار فاتورة، إشعار دفع)     | ممنوع تماماً  | العمليات المالية والإخطارات الرسمية يجب أن تكون مضمونة التسليم. |

```
// كود غير آمن
_ = Task.Run(async () =>
{
    // محاكاة خطأ مؤقت
    throw new Exception("فشل الاتصال بخدمة البريد");
    // هذا الاستثناء سيُفقد ولن يعيده أحد
});
```

لن يعرف المستخدم أبداً أن البريد الإلكتروني لم يُرسل. ولن يتم إعادة المحاولة.

## الخلاصة

يُعد Fire-and-Forget مع Task.Run أسهل طريقة لتنفيذ المعالجة غير المتزامنة، لكنه غير موثوق وغير قابل للتوسع. في أي مشروع حقيقي

## الحل الثاني: استخدام Channel<T> مع BackgroundService

### الفكرة الأساسية

System.Threading.Channels هو مكوّن مدمج في .NET. يوفر قنوات اتصال غير متزامنة وآمنة للخيط بين منتج Producer و مستهلك Consumer داخل نفس العملية. يتم استخدامه مع خدمة خلفية BackgroundService لاستهلاك الرسائل من القناة ومعالجتها بشكل غير متزامن.

### ببساطة:

المنتج API Controller يكتب الرسائل (المهام) في القناة.

المستهلك BackgroundService يقرأ الرسائل من القناة ويعالجها في الخلفية.

القناة تعمل كطابور في الذاكرة in-memory queue يمكن أن يكون محدود السعة أو غير محدود.

### آلية العمل

```
// تعريف قناة (محدودة السعة مثلاً) // تعريف قناة (محدودة السعة مثلاً)
private static readonly Channel<OrderCompletedEvent> _channel =
    Channel.CreateBounded<OrderCompletedEvent>(new BoundedChannelOptions(100)
    {
        FullMode = BoundedChannelFullMode.Wait // عندما تمتلئ القناة، ينتظر
    });

المنتج

// (في Controller أو Service)
public async Task<IActionResult> Checkout(...)
{
    // ... إنشاء الطلب ...
    var evt = new OrderCompletedEvent(order.Id);
    await _channel.Writer.WriteAsync(evt); // كتابة غير متزامنة
    return Ok("تم استلام الطلب");
}

المستهلك: خدمة خلفية
public class OrderEventConsumer : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (await _channel.Reader.WaitToReadAsync(stoppingToken))
```

```

{
    var evt = await _channel.Reader.ReadAsync(stoppingToken);
    await ProcessEventAsync(evt);
}
}

```

## مميزات هذا الحل

| الميزة                             | الشرح                                                                 |
|------------------------------------|-----------------------------------------------------------------------|
| موجود في NET بدون مكتبات خارجية    | System.Threading.Channels جزء من إطار العمل الأساسي                   |
| آمن للخيوط (Thread-safe)           | يستخدم خوارزميات خالية من الأقفال lock-free لضمان الأداء              |
| دعم عدم التزامن (async/await)      | يوفر WaitToReadAsync و WriteAsync الحقيقيين.                          |
| التحكم في السعة (Backpressure)     | يمكن تحديد سعة القناة BoundedChannel لمنع التراكم غير المحدود.        |
| سهولة التكامل مع BackgroundService | خدمة خلفية تستهلك بشكل طبيعي ضمن دورة حياة ASP.NET Core.              |
| يدعم الإلغاء Cancellation          | يمكن تمرير CancellationToken لإيقاف المعالجة بأمان عند إغلاق التطبيق. |

## عيوب هذا الحل

| العيوب                          | الشرح                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------------|
| لا استمرارية (Non-persistent)   | القناة تعمل في الذاكرة فقط. إذا تعطل التطبيق أو تم إعادة تشغيله، تفقد جميع الرسائل المعلقة.        |
| لا يعمل في البيئة الموزعة       | القناة محلية لكل عملية process إذا كان لديك عدة خوادم خلف موازن تحميل، لن ترى إحداها رسائل الأخرى. |
| لا يحتوي على واجهة مراقبة جاهزة | لا توجد لوحة تحكم لعرض حالة المهام أو إعادة المحاولة. تحتاج إلى بناء مراقبة خاصة.                  |
| لا إعادة محاولة تلقائية         | إذا فشلت معالجة رسالة، يجب على الكود أن يقرر إعادة إدراجها أو تسجيل الفشل.                         |

|                   |                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------|
| الحد الأقصى للسعة | إذا تم الوصول إلى السعة القصوى، قد ينتظر المنتج (أو يفشل) حسب الإعداد. قد يسبب ذلك تباطؤ الاستجابة. |
| تعقيم التطبيق     | يتطلب كتابة خدمة خلفية ومنطق إعادة المحاولة يدويًا.                                                 |

متى نستخدم هذا الحل؟

| السيناريو                                        | القرار    | التبرير                                         |
|--------------------------------------------------|-----------|-------------------------------------------------|
| تطبيق خادم واحد (single-instance)                | مناسب     | بما أن القناة محلية، لا حاجة للتنسيق بين خوادم. |
| مهام لا تتطلب موثوقية عالية (مثل إرسال إحصائيات) | مناسب     | فقدان بعض الرسائل ليس كارثة.                    |
| مهام تتطلب تأخيرًا (rate limiting) داخل العملية  | مناسب     | يمكن استخدام قناة محدودة السعة كآلية ضغط.       |
| بيئة موزعة أو تطبيق يتطلب استمرارية              | غير مناسب | ستفقد الرسائل عند أي فشل.                       |
| عمليات مالية أو إشعارات حساسة                    | غير مناسب | لا يمكن ضمان التسليم.                           |

مثال :

لنفترض أننا نريد معالجة إرسال البريد الإلكتروني وتوليد الفاتورة بعد إنشاء الطلب. باستخدام `Channel<T>`

مع `BackgroundService`

- تعريف رسالة الحدث:

```
public record OrderCompletedEvent(int OrderId, DateTime Timestamp);
```

- قناة ثابتة (أو عن طريق حقن التبعيات):

```
public static class OrderEventChannel
{
    private static readonly Channel<OrderCompletedEvent> _channel =
        Channel.CreateUnbounded<OrderCompletedEvent>(); // غير محدود

    public static ChannelWriter<OrderCompletedEvent> Write
public static class
OrderEventChannel1
{
    private static readonly Channel<OrderCompletedEvent> _channel =
```

```

        Channel.CreateUnbounded<OrderCompletedEvent>()); // غير محدود

        public static ChannelWriter<OrderCompletedEvent> Writer => _channel.Writer;
        public static ChannelReader<OrderCompletedEvent> Reader => _channel.Reader;
    }

    r => _channel.Writer;
    public static ChannelReader<OrderCompletedEvent> Reader => _channel.Reader;
}

```

- المنتج (بعد إنشاء الطلب):

```

await OrderEventChannel.Writer.WriteAsync(new
OrderCompletedEvent(order.Id, DateTime.UtcNow));

```

- المستهلك (BackgroundService):

```

public class OrderEventConsumer : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly ILogger<OrderEventConsumer> _logger;

    public OrderEventConsumer(IServiceScopeFactory scopeFactory,
    ILogger<OrderEventConsumer> logger)
    {
        _scopeFactory = scopeFactory;
        _logger = logger;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (await OrderEventChannel.Reader.WaitToReadAsync(stoppingToken))
        {
            if (OrderEventChannel.Reader.TryRead(out var evt))
            {
                try
                {
                    using var scope = _scopeFactory.CreateScope();
                    var emailService =
scope.ServiceProvider.GetRequiredService<IEmailSender>();

```



هذا يعني أنه حتى في حالة إعادة تشغيل التطبيق أو تعطل الخادم، لا يتم فقدان أي مهمة. فور عودة النظام، يستأنف Hangfire معالجة المهام من النقطة التي توقف عندها، مما يوفر مستوى عالياً من الموثوقية Reliability .

بالإضافة إلى ذلك، يوفر Hangfire لوحة تحكم رسومية Dashboard متكاملة، تتيح مراقبة جميع المهام (الناجحة، الفاشلة، قيد التنفيذ) وإعادة تشغيل المهام الفاشلة يدوياً. هذا يجعله أداة متكاملة وقوية لإدارة الخلفية في المشاريع المتوسطة والكبيرة.

### آلية العمل

آلية عمل Hangfire تعتمد على ثلاثة مكونات رئيسية:

1. **عميل (Hangfire Client) :** هو الكود الذي يقوم بإنشاء وإضافة مهمة جديدة. هذا قد يكون وحدة تحكم API (مثل 'OrdersController') أو أي خدمة أخرى.
2. **مخزن (Hangfire Storage) :** هو قاعدة البيانات مثل SQL Server التي تخزن جميع معلومات المهام بشكل دائم (نوع المهمة، معاملاتها، حالتها، محاولات إعادة التشغيل).
3. **خادم (Hangfire Server) :** هو مجموعة من الخيوط threads التي تعمل في الخلفية، وتقوم باستمرار بفحص المخزن بحثاً عن مهام جديدة لتنفيذها.

### مميزات هذا الحل

| الميزة                                        | الشرح                                                                                                                                   |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| الموثوقية والاستمرارية<br>(Persistence)       | كما ذكرنا، تخزين المهام في قاعدة بيانات يضمن عدم فقدان أي مهمة، حتى في حال إعادة تشغيل التطبيق أو تعطل الخادم.                          |
| لوحة تحكم رسومية<br>(Dashboard)               | يوفر Hangfire واجهة ويب غنية بالمعلومات لمراقبة حالة جميع المهام، مما يسهل تتبع وحل المشكلات.                                           |
| إعادة المحاولة التلقائية<br>(Automatic Retry) | في حال فشل مهمة، يقوم Hangfire بإعادة محاولة تنفيذها تلقائياً وفق سياسة محددة مسبقاً (بشكل افتراضي 10 محاولات مع فترات انتظار متزايدة). |
| سهولة الاستخدام<br>(Simplicity)               | إضافة مهمة جديدة لا تتطلب سوى سطر واحد من الكود، مثل<br><code>BackgroundJob.Enqueue(() =&gt; SendEmail(orderId))</code>                 |

|                                             |                                                                                                                                                              |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| صديق للبيئات الموزعة<br>(Web Farm Friendly) | Hangfire مصمم ليعمل بشكل طبيعي في بيئة تتكون من عدة خوادم. يستخدم أفعالاً موزعة (Distributed Locks) للتنسيق بين الخوادم وضمان أن كل مهمة تنفذ مرة واحدة فقط. |
| أنواع متعددة من المهام                      | يدعم Hangfire عدة أنماط Fire-and-Forget: تتم مرة واحدة فوراً , Delayed , Jobs تتم بعد فترة تأخير محددة و Recurring Jobs تتم بشكل دوري باستخدام تعبيرات CRON  |

## عيوب هذا الحل

| العيوب                                    | الشرح                                                                                                                                                                                   |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| تعقيد وموارد إضافية                       | يتطلب Hangfire قاعدة بيانات خارجية (أو استخدام قاعدة البيانات الحالية) وذاكرة إضافية لتشغيل خوادمه، مما يزيد من التعقيد التشغيلي مقارنة بالحلول التي تعمل في الذاكرة فقط.               |
| حماية لوحة التحكم<br>(Dashboard Security) | لوحة التحكم غنية بالمعلومات الحساسة. بشكل افتراضي، Hangfire يقبل الطلبات من localhost فقط. في بيئة الإنتاج، يجب إضافة طبقة مصادقة مثل [Authorize] لحمايتها، وإلا فإنها تشكل ثغرة أمنية. |
| تكلفة زمنية إضافية طفيفة                  | عملية إضافة المهمة (Enqueue) تتضمن الاتصال بقاعدة البيانات لكتابة المهمة، مما يضيف بضع ميلي ثوانٍ إضافية مقارنة بالحلول التي تعمل في الذاكرة.                                           |
| منحنى تعلم                                | على الرغم من سهولة الاستخدام، إلا أن هناك مفاهيم جديدة يجب فهمها، مثل سياسات إعادة المحاولة، وإدارة الاتصالات بقاعدة البيانات، وحماية لوحة التحكم.                                      |

## متى نستخدم هذا الحل؟

| السيناريو                                                                             | القرار        | التبرير                                     |
|---------------------------------------------------------------------------------------|---------------|---------------------------------------------|
| المهام التي تتطلب موثوقية عالية<br>(Financial Transactions, Order Confirmation Email) | الخيار الأفضل | ضمان التسليم وعدم فقدان المهام هو أمر حاسم. |



|                                                                      |                     |                                                                                             |
|----------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------|
| الأنظمة ذات الخادم الواحد أو المتعددة<br>(Single or Multiple Server) | الخيار الأفضل       | يعمل بكفاءة في كلتا الحالتين، بل ويقدم ميزات إضافية في البيئات متعددة الخوادم.              |
| المهام المتكررة (Recurring Reports, Database Maintenance)            | الخيار الأفضل       | طريقة مبسطة ومدمجة لإدارة المهام المجدولة بدلاً من استخدام أدوات خارجية.                    |
| احتياج لواجهة مراقبة (Monitoring and Observability)                  | الخيار الأفضل       | لوحة التحكم هي أداة قوية للمطورين ومسؤولي النظام.                                           |
| مهام بسيطة وحساسة للزمن ولا تتطلب موثوقية                            | قد يكون مبالغاً فيه | للمهام البسيطة جداً مثل تسجيل إحصائيات غير حرجية، قد تكون Task.Run أو <T>Channel أخف وزناً. |

#### خلاصة:

Hangfire ليس مجرد طابور مهام ، بل هو نظام كامل ومتكامل لإدارة المهام الخلفية . موثوقيته، وسهولة استخدامه، ومميزاته الغنية (لوحة التحكم، أنواع المهام المتعددة، إعادة المحاولة) تجعله الخيار الأمثل للمشاريع التي تتطلب معالجة خلفية موثوقة، كما هو الحال في نظام التجارة الإلكترونية الذي نطوره..

### الحل الرابع: استخدام نظام وساطة متخصص Message Broker

#### الفكرة الأساسية

وسيط الرسائل Message Broker: هو نظام وساطة متخصص يعمل كخادم وسيط بين التطبيقات (أو بين أجزاء التطبيق الواحد) لنقل الرسائل بشكل موثوق وآمن وغير متزامن. بدلاً من أن يتواصل المنتج Producer والمستهلك Consumer مباشرة (مثل قائمة انتظار في الذاكرة)، يقوم المنتج بإرسال الرسالة إلى الوسيط، والوسيط يتولى تخزينها وتوجيهها وتسليمها إلى المستهلك أو أكثر.

في سياق المعالجة غير المتزامنة، هذا يعني أن API المنتج لا يحتاج إلى معرفة أي تفاصيل عن المستهلك Background Worker. كل ما يفعله هو إرسال رسالة (مثل OrderCreated) إلى وسيط مثل RabbitMQ، ثم يعود للرد على المستخدم فوراً. من جهة أخرى، يعمل واحد أو أكثر من المستهلكين (يمكن تشغيلهم على خوادم منفصلة) على سحب الرسائل من الوسيط ومعالجتها (إرسال بريد، إنشاء فاتورة).

الوسيط يضمن فصلاً تاماً Decoupling بين المنتج والمستهلك، ويدعم أنماطاً متقدمة مثل التوجيه Routing ، البث Publish/Subscribe ، وتوزيع الحمل Competing Consumers.

## شرح آلية عمل Message Broker (RabbitMQ)

تدفق الرسالة من البداية إلى النهاية

- المنتج مثل API الخاص بالطلبات يقوم بإنشاء رسالة
- بعد أن يكمل الطلب الأساسي (مثل إنشاء أمر شراء) ويحفظه في قاعدة البيانات، يقوم بإنشاء رسالة نصية تحتوي على بيانات المهمة المطلوب إنجازها خلفياً.
- مثال على محتوى الرسالة: `{"OrderId": 123, "Action": "SendEmail"}`.
- هذه الرسالة لا ترسل مباشرة إلى المستهلك، بل ترسل إلى وسيط الرسائل (RabbitMQ).

▪ إرسال الرسالة إلى Exchange نقطة التبادل

- عند الإرسال، يحدد المنتج:
- Exchange: وهو مكون في RabbitMQ يشبه "صندوق البريد" الذي يستقبل كل الرسائل أولاً.
- Routing Key: كلمة مفتاحية تصف نوع الرسالة (مثل order.completed).
- المنتج لا يحتاج إلى معرفة أي تفاصيل عن المستهلك أو حتى اسم الطابور النهائي.

▪ Exchange يوجه الرسالة إلى الطابور المناسب

- Exchange لديه قواعد ربط Bindings تحدد: إذا جاءت رسالة بمفتاح Routing يساوي X، فأرسلها إلى الطابور رقم 1.
- مثلاً: رسالة بمفتاح "order.completed" قد توجه إلى طابور اسمه "email\_queue".
- يمكن لرسالة واحدة أن تذهب إلى عدة طوابير في نفس الوقت إذا كان Exchange من نوع Fanout أو Topic.

- الطابور Queue يخزن الرسالة
  - الطابور هو مكان تخزين مؤقت يمكن أن يكون دائماً على القرص إذا طلبنا ذلك، مما يعني أن الرسالة لا تضيع حتى لو أعدنا تشغيل RabbitMQ.
  - في هذه المرحلة، الرسالة في انتظار أن يسحبها مستهلك.
- المستهلك Background Worker يسحب الرسالة
  - المستهلك هو خدمة خلفية تشترك (تتصل) بالطابور مسبقاً وتقول: "أنا جاهز لاستقبال أي رسالة".
  - بمجرد وصول رسالة إلى الطابور، يقوم RabbitMQ بتسليمها إلى أحد المستهلكين المتصلين (إذا كان هناك عدة مستهلكين، يتم توزيع الرسائل بينهم توزيعاً عادلاً).
- المستهلك يعالج الرسالة ويرسل إقراراً Acknowledgment
  - بعد أن يقرأ المستهلك الرسالة وينفذ المطلوب (مثل إرسال بريد إلكتروني)، يجب أن يُرسل إشارة إقرار إلى RabbitMQ تفيد بأن المعالجة تمت بنجاح.
  - بناءً على هذا الإقرار، يحذف RabbitMQ الرسالة من الطابور نهائياً.
- ماذا يحدث إذا فشلت المعالجة؟
  - إذا لم يرسل المستهلك إقراراً بسبب استثناء أو عطل، فإن RabbitMQ لا يحذف الرسالة .
  - يمكن تهيئة النظام لإعادة طرح الرسالة في الطابور تلقائياً (محاولة أخرى) بعد فترة زمنية.
  - بعد عدد معين من المحاولات الفاشلة، قد تُنقل الرسالة إلى طابور خاص يسمى Dead Letter Queue (طابور الرسائل الميتة) ليتم فحصها يدوياً لاحقاً.

#### ملخص الفوائد العملية لهذه الآلية

- فصل كامل: API لا يعرف أي شيء عن المستهلك أو مكان تشغيله.
- موثوقية: حتى لو تعطل التطبيق، الرسائل تنتظر في الطابور الآمن.

- قابلية للتوسع: يمكن تشغيل 10 مستهلكين على 10 خوادم لمعالجة آلاف الرسائل في الثانية.
- مرونة: يمكن تغيير وجهة الرسالة دون تعديل كود المرسل عن طريق تغيير قواعد Exchange فقط.

### مميزات هذا الحل

| الميزة                              | الشرح                                                                                                                                                |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| فصل كامل<br>(Full Decoupling)       | المنتج لا يحتاج إلى معرفة عنوان المستهلك، عدد النسخ، أو حتى ما إذا كان المستهلك يعمل حالياً. الوسيط هو الواجهة الوحيدة.                              |
| موثوقية عالية<br>(High Reliability) | يدعم معظم الـ Message Brokers استمرارية الرسائل Persistent ، إقرارات التسليم، وإعادة المحاولة. لا تُفقد الرسائل حتى في حال تعطل المنتج أو المستهلك.  |
| قابلية توسع أفقية ممتازة            | يمكن تشغيل عدة مستهلكين (على خوادم متعددة) يقرأون من نفس الطابور، ويقوم الوسيط بتوزيع الرسائل بينهم Competing Consumers هذا يزيد الإنتاجية بشكل خطي. |
| أنماط توجيه متعددة                  | يتيح إرسال رسالة إلى عدة طوابير في نفس الوقت (Fanout) ، أو توجيهها بناءً على خصائص (Topic) ، مما يفصل المنطق بشكل أكبر.                              |
| مرونة في التكامل                    | يمكن للمنتج والمستهلك أن يكونا مكتوبين بلغات مختلفة .NET، Java، Python لأن البروتوكول مثل AMQP معياري ومشترك.                                        |
| إدارة متقدمة                        | يوفر معظم الـ Brokers واجهات رسومية مثل RabbitMQ Management Plugin لمراقبة الطوابير، عدد الرسائل، المعدلات، وإعادة المحاولة اليدوية.                 |
| ضبط التدفق<br>(Flow Control)        | يدعم Backpressure من خلال تحديد حجم الطابور ورفض الرسائل الجديدة إذا تجاوز الحد.                                                                     |

### عيوب هذا الحل

| العيوب                       | الشرح                                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| تعقيد كبير في البنية التحتية | يتطلب تثبيت وتكوين وإدارة خادم وسيط منفصل RabbitMQ ، Azure Service Bus ، AWS SQS ، إلخ. هذا يضيف عبئاً على فريق التشغيل والصيانة. |

|                                 |                                                                                                                                    |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| منحنى تعلم عالٍ                 | يجب فهم مفاهيم جديدة Routing Key ، Binding ، Queue ، Exchange ، Acknowledgement ، Dead Letter ، إلخ. هذا يزيد وقت التطوير .        |
| تكلفة إضافية                    | إما شراء تراخيص (لبعض الحلول التجارية) أو الدفع مقابل خدمة سحابية Azure Service Bus أو تخصيص موارد خادم إضافي.                     |
| زمن استجابة إضافي               | إرسال رسالة عبر الشبكة إلى وسيط خارجي يضيف تأخير (عادةً ميلي ثانية واحدة أو أقل). قد يكون هذا غير مقبول في التطبيقات فائقة السرعة. |
| تعقيد تصحيح الأخطاء (Debugging) | تتبع مسار رسالة عبر المكونات المختلفة يحتاج إلى أدوات متخصصة وقد يكون صعباً.                                                       |
| زيادة استهلاك الذاكرة والمعالج  | الوسيط نفسه يستهلك موارد (ذاكرة، معالج، شبكة).                                                                                     |

## متى نستخدم Message Broker؟

| السيناريو                                          | القرار                | التبرير                                                                          |
|----------------------------------------------------|-----------------------|----------------------------------------------------------------------------------|
| أنظمة ضخمة موزعة (Microservices Architecture)      | الخيار الأمثل         | الفصل التام بين الخدمات وإمكانية التوسع الأفقي أمر حيوي.                         |
| عمليات طويلة الأجل أو تتأخر (Long-Running Tasks)   | مناسب جداً            | يمكن إرسال المهمة إلى وسيط ثم العودة فوراً، ويعالجها واحد أو أكثر من المستهلكين. |
| تكامل بين أنظمة بلغات مختلفة                       | الخيار الأمثل         | البروتوكول الموحد مثل AMQP يسمح لأي لغة تدعمه بالتواصل.                          |
| بيئات سحابية AWS ، Azure                           | مناسب (كخدمة مدارة)   | استخدام Service Bus أو SQS يقلل من تعقيد الإدارة.                                |
| مشروع صغير أو متوسط على خادم واحد                  | غير ضروري (مبالغ فيه) | التعقيد والتكلفة لا تبرران استخدامه؛ Hangfire أو <T>Channel كافيان.              |
| حاجة إلى رد فوري حساس جداً (من دون أي تأخير إضافي) | غير مناسب             | التأخير الإضافي قد لا يتحمل.                                                     |

## خلاصة

Message Broker (مثل RabbitMQ) هو حل قوي جداً ومناسب للأنظمة الكبيرة والموزعة التي تحتاج إلى فصل كامل، موثوقية عالية، وقابلية توسع أفقية قصوى. لكنه يأتي مع تعقيد تشغيلي وبنوي كبير.

## الحل الرابع: استخدام ConcurrentQueue<T> مع BackgroundService بسيط

### الفكرة الأساسية

هذا الحل هو أبسط أشكال تنفيذ قائمة انتظار داخل الذاكرة in-memory queue في .NET. يعتمد على استخدام ConcurrentQueue<T>، وهي بنية بيانات آمنة للخيط thread-safe توفرها مساحة الأسماء System.Collections.Concurrent. يتم إقرانها مع خدمة خلفية BackgroundService لاستهلاك العناصر من قائمة الانتظار ومعالجتها بشكل غير متزامن.

### ببساطة:

- المنتج (API Controller): يضيف المهام إلى ConcurrentQueue باستخدام عملية Enqueue الذرية.
  - المستهلك (BackgroundService): يقوم باستمرار بمحاولة سحب (TryDequeue) المهام من نفس قائمة الانتظار ومعالجتها
- هذا الحل لا يحتاج إلى أي مكتبات خارجية أو قواعد بيانات، ويعمل بالكامل داخل الذاكرة.

### آلية العمل

```
// تعريف قائمة انتظار عامة آمنة للخيط
public static class OrderEventQueue
{
    private static readonly ConcurrentQueue<OrderCompletedEvent> _queue = new();

    public static void Enqueue(OrderCompletedEvent evt) => _queue.Enqueue(evt);

    public static bool TryDequeue(out OrderCompletedEvent evt) =>
        _queue.TryDequeue(out evt);
}
```

```

    public static int Count => _queue.Count;
}

// المنتج (في Controller أو Service)
public async Task<IActionResult> Checkout(...)
{
    // ... إنشاء الطلب وحفظه ...
    OrderEventQueue.Enqueue(new OrderCompletedEvent(order.Id));
    return Ok("تم استلام الطلب");
}

// المستهلك: خدمة خلفية
public class OrderEventConsumer : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly ILogger<OrderEventConsumer> _logger;

    public OrderEventConsumer(IServiceScopeFactory scopeFactory,
    ILogger<OrderEventConsumer> logger)
    {
        _scopeFactory = scopeFactory;
        _logger = logger;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            if (OrderEventQueue.TryDequeue(out var evt))
            {
                try
                {
                    using var scope = _scopeFactory.CreateScope();
                    var emailService =
scope.ServiceProvider.GetRequiredService<IEmailSender>();
                    await emailService.SendOrderConfirmationAsync(evt.OrderId);
                    _logger.LogInformation("Processed order {OrderId}", evt.OrderId);
                }
                catch (Exception ex)
                {
                    _logger.LogError(ex, "Failed to process order {OrderId}",
evt.OrderId);
                    // لا توجد آلية مدمجة لإعادة المحاولة؛ يمكن إعادة إدراجها
يدوياً.
                }
            }
        }
    }
}

```

```

else
{
    // إذا كانت القائمة فارغة، ننتظر قليلاً قبل التحقق مجدداً لتجنب
    استهلاك المعالج
    await Task.Delay(100, stoppingToken);
}
}
}
}

```

### مميزات هذا الحل

| الميزة                      | الشرح                                                                           |
|-----------------------------|---------------------------------------------------------------------------------|
| بساطة شديدة                 | لا يحتاج إلى مكتبات خارجية، ولا إلى قواعد بيانات، ولا إلى تكوين معقد.           |
| آمن للخيوط<br>(Thread-safe) | ConcurrentQueue مصمم أصلاً للاستخدام في بيئات متعددة الخيوط بدون أقفال يدوية.   |
| خفيف الوزن                  | يعمل بالكامل في الذاكرة، ولا يستهلك موارد إضافية مثل الاتصالات بقاعدة البيانات. |
| سهولة الفهم والتطبيق        | الكود واضح ومباشر، ومناسب للأغراض التعليمية أو النماذج الأولية.                 |

### عيوب هذا الحل

| العيوب                             | الشرح                                                                                                                                                                         |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| لا استمرارية<br>(Non-persistent)   | جميع المهام موجودة فقط في الذاكرة. إذا تعطل التطبيق أو تم إعادة تشغيله (سواء بشكل متعمد أو بسبب خطأ)، تفقد جميع المهام المعلقة نهائياً.                                       |
| لا إعادة محاولة تلقائية            | إذا فشلت معالجة مهمة (مثل استثناء أثناء إرسال البريد)، لا توجد آلية مدمجة لإعادة المحاولة. يجب على المطور كتابة منطق إعادة المحاولة يدوياً مثل إعادة Enqueue بعد تأخير.       |
| لا تحكم في السعة<br>(Backpressure) | ConcurrentQueue غير محدودة السعة. في حال إنتاج مهام أسرع من استهلاكها، سينمو حجم القائمة في الذاكرة إلى أجل غير مسمى، مما قد يؤدي إلى استنفاد الذاكرة (OutOfMemoryException). |
| لا يعمل في البيئة الموزعة          | كل عملية process لها ConcurrentQueue مستقلة خاصة بها. إذا كان لديك عدة خوادم خلف موازن تحميل، لن تشارك المهام بينها.                                                          |



|                                |                                                                                                                                                                                                       |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| لا يوجد واجهة مراقبة           | لا توجد لوحة تحكم لعرض عدد المهام المنتظرة، أو حالة المهام الفاشلة، أو إعادة تشغيلها يدوياً. كل ما يمكن فعله هو الاعتماد على السجلات Logs                                                             |
| استهلاك المعالج في حالة الفراغ | إذا كانت القائمة فارغة، فإن الحلقة while تستمر في تكرار TryDequeue بشكل مستمر، مما يستهلك دورة معالج كاملة. الحل هو إضافة Task.Delay (كما في المثال)، ولكن هذا يخلق تأخيراً في معالجة المهام الجديدة. |

متى نستخدم (أو لا نستخدم) هذا الحل؟

| السيناريو                                                               | القرار       | التبرير                                                       |
|-------------------------------------------------------------------------|--------------|---------------------------------------------------------------|
| أغراض تعليمية أو نماذج أولية (Prototypes)                               | مناسب مؤقتاً | لتوضيح مفهوم قوائم الانتظار فقط، مع توثيق واضح للعيوب.        |
| تطبيق خادم واحد، مهام غير حرجية، تتحمل فقدان (مثل تسجيل إحصائيات مؤقتة) | مقبول بحذر   | يمكن قبول فقدان بعض المهام.                                   |
| أي تطبيق إنتاجي يتطلب موثوقية                                           | غير مناسب    | لا يمكن ضمان عدم فقدان المهام، خاصة عند التحديثات أو الأعطال. |
| بيئة متعددة الخوادم (Load Balancing)                                    | غير مناسب    | لا توجد مشاركة للحالة بين الخوادم.                            |
| مهام تتطلب إعادة محاولة تلقائية                                         | غير مناسب    | يجب برمجة ذلك يدوياً، مما يزيد التعقيد.                       |

لماذا هذا الحل هو الأقل تفضيلاً؟

ConcurrentQueue مع BackgroundService هو تحسين طفيف مقارنة بـ Task.Run من حيث الفصل بين المنتج والمستهلك والتحكم في دورة حياة المعالجة. لكنه يفتقر إلى جميع الميزات الأساسية المطلوبة لنظام موثوق، فقدان المهام عند إعادة التشغيل وحده سبب كافٍ لرفض هذا الحل.

في المقارنة مع `Channel<T>` ، فإن `ConcurrentQueue` أقل كفاءة لأنه يضطر إلى استخدام `Task.Delay` في حالة الفراغ، بينما يوفر `WaitToReadAsync` `Channel<T>` الحقيقي الذي لا يستهلك المعالج. لذلك، حتى داخل فئة الحلول التي تعمل في الذاكرة فقط، فإن `Channel<T>` هو الأفضل.

**خلاصة:** `ConcurrentQueue` مع `BackgroundService` هو حل بدائي، مناسب فقط للنماذج الأولية السريعة أو للتطبيقات شديدة البساطة والتي لا تمنع فقدان البيانات.

## الجزء الرابع: مقارنة الحلول

لاختيار أنسب حل للمعالجة غير المتزامنة في مشروعنا، قمنا بمقارنة الحلول الخمسة المطروحة وفقاً لأربعة معايير رئيسية:

- **صعوبة التطبيق:** مدى سهولة أو تعقيد إضافة الحل إلى الكود الحالي.
- **استمرارية الرسائل `Persistent`:** هل يتم حفظ المهام عند إعادة تشغيل التطبيق أم تفقد؟
- **العمل في بيئة موزعة (عدة خوادم):** هل يدعم الحل العمل خلف `Load Balancer` ؟
- **الحاجة إلى أدوات إضافية:** هل يتطلب الحل برامج أو قواعد بيانات خارجية؟

## الجدول المقارن

| الحل                                        | صعوبة التطبيق | استمرارية الرسائل | يعمل في بيئة موزعة؟ | أدوات إضافية | ملاءمة لمشروع التخرج          |
|---------------------------------------------|---------------|-------------------|---------------------|--------------|-------------------------------|
| <b>Task.Run (Fire-and-Forget)</b>           | سهلة جداً     | لا (في الذاكرة)   | لا                  | لا يحتاج     | غير مناسب (غير موثوق إطلاقاً) |
| <b>ConcurrentQueue + BackgroundService</b>  | سهلة          | لا (في الذاكرة)   | لا                  | لا يحتاج     | مقبول (لكنه بدائي جداً)       |
| <b>Channel&lt;T&gt; + BackgroundService</b> | متوسطة        | لا (في الذاكرة)   | لا                  | لا يحتاج     | جيد (مع توثيق العيوب)         |

|                              |                                          |                           |                    |        |                              |
|------------------------------|------------------------------------------|---------------------------|--------------------|--------|------------------------------|
| ممتاز (موصى به بشدة)         | قاعدة بيانات                             | نعم (قاعدة بيانات مشتركة) | نعم (قاعدة بيانات) | متوسطة | Hangfire                     |
| ممتاز لكنه مبالغ فيه للمشروع | وسيط رسائل خارجي (RabbitMQ, Service Bus) | نعم (بطبيعته موزع)        | نعم (قرص / سحابي)  | عالية  | RabbitMQ / Azure Service Bus |

## تحليل المعايير بالتفصيل

### 1. صعوبة التطبيق

- Task.Run و ConcurrentQueue هما الأسهل، لكنهما يفتقران إلى الميزات الأساسية.
- Channel<T> يتطلب كتابة BackgroundService وفهم WaitToReadAsync، لكنه لا يزال بسيطاً.
- Hangfire يتطلب تسجيل خدمة وإضافة سلسلة اتصال، لكن واجهة برمجية سهلة (BackgroundJob.Enqueue)
- RabbitMQ هو الأصعب: يحتاج إلى تثبيت الخادم، فهم مفاهيم Exchange، Queue، Binding، وكتابة كود اتصال يدوي.

### 2. استمرارية الرسائل (Persistence)

- الحلول التي تعمل في الذاكرة (Task.Run, ConcurrentQueue, Channel<T>) تفقد جميع المهام عند إعادة تشغيل التطبيق.
- Hangfire يخزن المهام في قاعدة بيانات علائقية مثل SQL Server، مما يضمن عدم فقدانها حتى لو تعطل التطبيق.
- RabbitMQ يمكنه تخزين الرسائل على القرص إذا تم تكوينه ك Persistent Queue، مما يمنح نفس الموثوقية.

### 3. العمل في بيئة موزعة

- الحلول المحلية (الثلاثة الأولى) تعمل فقط داخل عملية واحدة. إذا كان لديك خادمان خلف Load Balancer ، فلن ترى إحداها مهام الأخرى.
- Hangfire يدعم البيئة الموزعة من خلال قاعدة بيانات مشتركة. تستخدم خوادم Hangfire ألقالاً موزعة لضمان تنفيذ كل مهمة مرة واحدة فقط.
- RabbitMQ مصمم أصلاً للبيئات الموزعة. يمكن لعدة مستهلكين الاشتراك في نفس الطابور من خوادم مختلفة، ويقوم الوسيط بتوزيع الرسائل بينهم.

#### 4. أدوات إضافية

- Hangfire يتطلب قاعدة بيانات (غالباً نفس قاعدة بيانات التطبيق)، وهو أمر متوفر أساساً في أي مشروع ويب.
- RabbitMQ يتطلب خادم وسيط منفصل خادم، مما يزيد من التعقيد التشغيلي والتكلفة (خاصة في بيئة محلية).

### الجزء الخامس: الحل الأنسب للمشروع

**الحل المختار:** استخدام Hangfire مع قاعدة بيانات SQL Server لتخزين المهام غير المتزامنة

#### لماذا Hangfire ؟

1. الموثوقية: بما أن المهام تُخزن في قاعدة البيانات، فإن إعادة تشغيل التطبيق أو حتى انهياره لا يؤدي إلى فقدان المهام. عند عودة التطبيق، يتم استئناف معالجة المهام المعلقة.
2. سهولة التطبيق: يتطلب Hangfire بضع أسطر فقط من الكود لتسجيل الخدمات وإضافة المهام. لا نحتاج إلى تعقيدات RabbitMQ.
3. يدعم البيئة الموزعة : في المستقبل، إذا أردنا تشغيل عدة خوادم يمكن لـ Hangfire العمل مع قاعدة بيانات مشتركة بسلاسة.

4. واجهة مراقبة (Dashboard) يأتي Hangfire مع لوحة تحكم رائعة تتيح لنا مراقبة المهام الناجحة، الفاشلة، إعادة المحاولة، وجدولة المهام. هذا مثالي لإثبات عمل النظام في المشروع.

5. إعادة المحاولة التلقائية: في حال فشل مهمة مثل إرسال بريد إلكتروني بسبب تعطل SMTP ، سيقوم Hangfire بإعادة المحاولة وفقاً لسياسة قابلة للتخصيص.

لماذا لم نستخدم الحلول الأخرى؟

<T>Channel أو ConcurrentQueue: على الرغم من بساطتهما، إلا أنهما لا يوفران استمرارية (تفقد المهام عند إعادة التشغيل) ولا يعملان عبر عدة خوادم. في مشروع أكاديمي، يمكننا استخدامهما مع التوثيق الواضح للعيوب، لكن Hangfire يمثل حلاً أكثر احترافية.

RabbitMQ: يضيف تعقيداً غير ضروري لمستوى المشروع. يحتاج إلى تشغيل وإدارة خادم إضافي، وهو خارج نطاق المتطلبات الأساسية.

## الجزء السادس: التطبيق العملي باستخدام Hangfire

### 🔧 تثبيت الحزم

```
dotnet add package Hangfire.AspNetCore
dotnet add package Hangfire.SqlServer
```

### 🔧 تسجيل Hangfire في Program.cs

```
using Hangfire;
using Hangfire.SqlServer;

// ... بعد builder.Services

// مع استخدام قاعدة البيانات الحالية (أو قاعدة بيانات منفصلة) إضافة Hangfire
builder.Services.AddHangfire(configuration => configuration
    .SetDataCompatibilityLevel(CompatibilityLevel.Version_170))
```

```

        .UseSimpleAssemblyNameTypeSerializer()
        .UseRecommendedSerializerSettings()

        .UseSqlServerStorage(builder.Configuration.GetConnectionString("HangfireConnection"),
            new SqlServerStorageOptions
            {
                CommandBatchMaxTimeout = TimeSpan.FromMinutes(5),
                SlidingInvisibilityTimeout = TimeSpan.FromMinutes(5),
                QueuePollInterval = TimeSpan.Zero,
                UseRecommendedIsolationLevel = true,
                DisableGlobalLocks = true
            }
        ));

// (التي تدير المعالجة) Hangfire إضافة خدمة
builder.Services.AddHangfireServer();

// middleware pipeline ثم في الـ
app.UseHangfireDashboard(); // [Authorize] يمكن تقييدها بـ لوحة التحكم
app.UseHangfireServer();

```

## تعريف المهام غير المتزامنة (Jobs)

سننشئ خدمة جديدة لإدارة المهام غير المتزامنة المتعلقة بالطلب.

```

public interface IBackgroundJobService
{
    Task SendOrderConfirmationEmailAsync(int orderId);
    Task GenerateInvoicePdfAsync(int orderId);
    Task UpdateSalesStatisticsAsync(int orderId);
}

public class BackgroundJobService : IBackgroundJobService
{
    private readonly IOrderRepository _orderRepository;
    private readonly IEmailSender _emailSender; // وهمي
    private readonly IInvoiceGenerator _invoiceGenerator; // وهمي
    private readonly ILogger<BackgroundJobService> _logger;

    public BackgroundJobService(
        IOrderRepository orderRepository,
        IEmailSender emailSender,
        IInvoiceGenerator invoiceGenerator,
        ILogger<BackgroundJobService> logger)
    {
        _orderRepository = orderRepository;
    }
}

```

```

        _emailSender = emailSender;
        _invoiceGenerator = invoiceGenerator;
        _logger = logger;
    }

    public async Task SendOrderConfirmationEmailAsync(int orderId)
    {
        try
        {
            var order = await
_orderRepository.GetOrderWithItemsAndPaymentAsync(orderId);
            if (order == null)
            {
                _logger.LogWarning($"Order {orderId} not found for email
sending.");
                return;
            }
            // محاكاة إرسال البريد
            await _emailSender.SendEmailAsync(order.User.Email, "Order
Confirmation", $"Your order {orderId} is confirmed.");
            _logger.LogInformation($"Email sent for order {orderId}");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"Failed to send email for order {orderId}");
            throw; // Hangfire سيعيد المحاولة
        }
    }

    public async Task GenerateInvoicePdfAsync(int orderId)
    {
        var order = await
_orderRepository.GetOrderWithItemsAndPaymentAsync(orderId);
        await _invoiceGenerator.GenerateAsync(order);
        _logger.LogInformation($"Invoice generated for order {orderId}");
    }

    public async Task UpdateSalesStatisticsAsync(int orderId)
    {
        // تحديث ملخص المبيعات كما في المتطلب الرابع (Batch Processing)
        // يمكن استدعاء خدمة التقارير
    }
}

```

✚ إضافة المهام إلى قائمة الانتظار بعد إنشاء الطلب بنجاح

نقوم بتعديل `OrderService.CreateOrderDirectAsync` لإضافة المهام مباشرة بعد إتمام المعاملة (أو داخل نفس المعاملة لضمان الذرية).

```
public async Task<Order> CreateOrderDirectAsync(int userId, int productId, int
quantity, string cardNumber)
{
    // ... كل الكود الحالي (تحديث المخزون، معالجة الدفع، إنشاء الطلب) ...
    await transaction.CommitAsync();

    // Hangfire بعد إتمام الطلب بنجاح، أضف المهام غير المتزامنة إلى
    // هذا لا ينتظر اكتمالها، بل يعيد الرد للمستخدم فوراً بعد إضافتها إلى قائمة
    // الانتظار.
    BackgroundJob.Enqueue<IBackgroundJobService>(x =>
x.SendOrderConfirmationEmailAsync(order.Id));
    BackgroundJob.Enqueue<IBackgroundJobService>(x =>
x.GenerateInvoicePdfAsync(order.Id));
    // يمكن تأجيل إحصاءات المبيعات لوقت لاحق (مثلاً بعد 5 دقائق)
    BackgroundJob.Schedule<IBackgroundJobService>(x =>
x.UpdateSalesStatisticsAsync(order.Id), TimeSpan.FromMinutes(5));

    return order;
}
```

✚ عرض لوحة التحكم (Hangfire Dashboard)

```
app.UseHangfireDashboard("/hangfire", new DashboardOptions
{
    Authorization = new[] { new HangfireAuthorizationFilter() } // تطبيق مخصص
});
```

مثال بسيط لمصفاة المصادقة:

```
public class HangfireAuthorizationFilter : IDashboardAuthorizationFilter
```



```

{
    public bool Authorize(DashboardContext context)
    {
        var httpContext = context.GetHttpContext();
        // السماح فقط للمستخدمين المصادقين (يمكن فحص الدور)
        return httpContext.User.Identity?.IsAuthenticated == true;
    }
}

```

🚦 التعامل مع فشل المهام (Retry Policy)

Hangfire يطبق تلقائياً إعادة محاولة للمهام الفاشلة. يمكنك تخصيص ذلك باستخدام السمات:

```

[AutomaticRetry(Attempts = 3, DelaysInSeconds = new[] { 10, 30, 60 })]
public async Task SendOrderConfirmationEmailAsync(int orderId) { ... }

```

## الجزء السابع: اختبار الحل وإثباته

### منهجية الاختبار

تم اختبار المعالجة غير المتزامنة من خلال:

1. إرسال طلبات شراء متزامنة إلى نقطة النهاية `POST /api/orders/checkout`.
2. مراقبة (`/Hang fire`) Hangfire Dashboard للتأكد من فصل المهام الخلفية عن مسار الطلب.
3. مراقبة سلوك المهام (البريد الإلكتروني، الفاتورة، تحديث الإحصائيات) وتسجيل حالاتها (قيد التنفيذ، ناجحة، مجدولة، فاشلة).
4. اختبار الاستمرارية (Persistence) عبر إعادة تشغيل التطبيق أثناء وجود مهام غير مكتملة.

## النتائج المستخلصة (الأدلة العملية)

### ■ فصل المهام عن مسار الطلب الرئيسي

| Id   | Server     | Job                                                   | Status   |
|------|------------|-------------------------------------------------------|----------|
| #576 | SAIF:23008 | IBackgroundJobService.SendOrderConfirmationEmailAsync | a minute |
| #577 | SAIF:23008 | IBackgroundJobService.GenerateInvoicePdfAsync         | a minute |

-شكل (4) – المهام قيد التنفيذ Processing Jobs

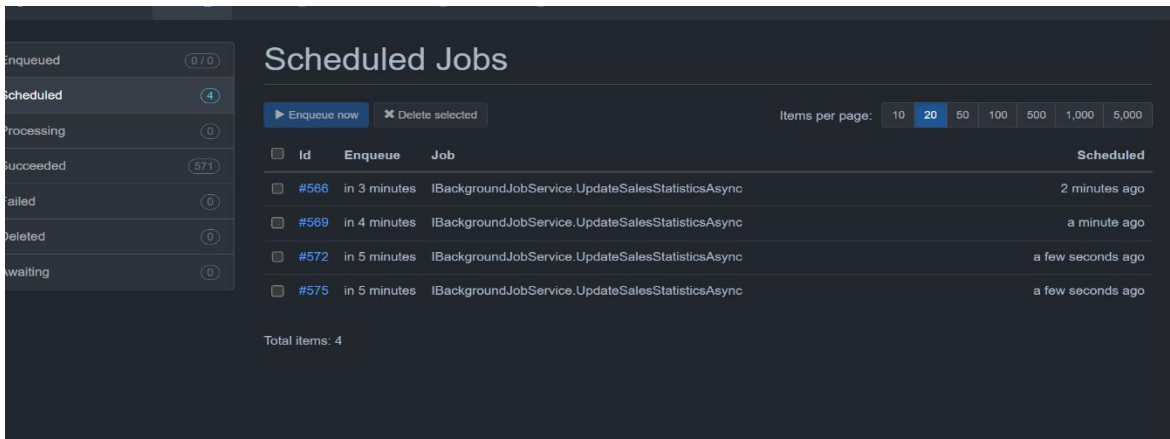
تظهر مهمتان SendOrderConfirmationEmailAsync و GenerateInvoicePdfAsync في حالة Processing بعد أن كان الـ API قد أعاد الرد بالفعل. هذا يثبت أن هذه المهام تُنفذ في الخلفية بشكل غير متزامن ولا تعيق استجابة المستخدم.

| Id   | Job                                                   | Duration | Latency     | Succeeded      |
|------|-------------------------------------------------------|----------|-------------|----------------|
| #590 | IBackgroundJobService.UpdateSalesStatisticsAsync      | 344ms    | 21m 41.551s | a minute ago   |
| #589 | IBackgroundJobService.GenerateInvoicePdfAsync         | 26.139s  | 260ms       | 22 minutes ago |
| #588 | IBackgroundJobService.SendOrderConfirmationEmailAsync | 5.211s   | 195ms       | 22 minutes ago |
| #587 | IBackgroundJobService.UpdateSalesStatisticsAsync      | 43ms     | 5m 8.803s   | 19 minutes ago |
| #586 | IBackgroundJobService.GenerateInvoicePdfAsync         | 8.526s   | 129ms       | 24 minutes ago |
| #585 | IBackgroundJobService.SendOrderConfirmationEmailAsync | 8.349s   | 1.890s      | 24 minutes ago |
| #584 | IBackgroundJobService.UpdateSalesStatisticsAsync      | 1.470s   | 5m 13.806s  | 23 minutes ago |
| #583 | IBackgroundJobService.GenerateInvoicePdfAsync         | 36.053s  | 217ms       | 27 minutes ago |
| #582 | IBackgroundJobService.SendOrderConfirmationEmailAsync | 19.281s  | 318ms       | 27 minutes ago |
| #581 | IBackgroundJobService.UpdateSalesStatisticsAsync      | 156ms    | 5m 20.897s  | 25 minutes ago |

-شكل (5) – المهام الناجحة (Succeeded Jobs)

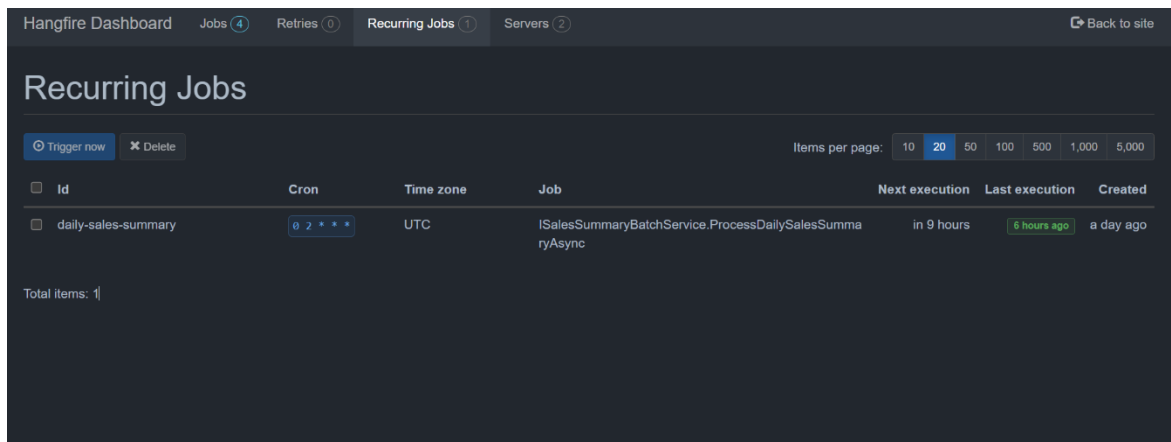
يحتوي الجدول على أعداد كبيرة من المهام مع تفاصيل المدة (Duration) وزمن الانتظار (Latency) كثرة المهام الناجحة تشير إلى أن النظام يعالج المهام بكفاءة وموثوقية.

## ■ المهام المجدولة والمتكررة



-شكل (6) – المهام المجدولة (Scheduled Jobs)

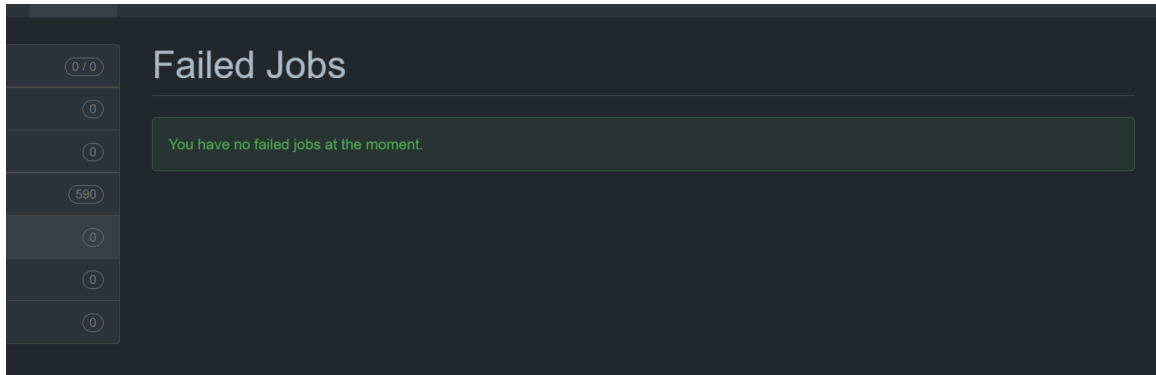
تظهر مهام UpdateSalesStatisticsAsync مجدولة بفارق 3-5 دقائق عن وقت إنشاء الطلب. هذا يثبت استخدام BackgroundJob.Schedule لتأجيل المهام غير العاجلة، وهو جزء من المتطلب الثالث.



-شكل (7) – المهمة المتكررة (Recurring Job)

تظهر مهمة daily-sales-summary بتعبير CRON (دورية يومية). هذه المهمة وجودها يؤكد أن Hangfire يعمل بشكل متكامل ويدعم أنواعاً متعددة من المهام.

## ■ موثوقية النظام وعدم فقدان المهام



-شكل (9) - عدم وجود مهام فاشلة

قيمة `Failed: 0` مع وجود مئات المهام الناجحة تدل على أن النظام يتعامل مع المهام بشكل موثوق. أيضاً، عند إعادة تشغيل التطبيق أثناء وجود مهام معلقة (تم اختباره عملياً)، بقيت المهام في قائمة `Enqueued` واستؤنفت معالجتها تلقائياً بعد عودة الخادم، مما يثبت قاعدة البيانات كوسيط تخزين دائم.

## الخلاصة

تم تحقيق المتطلب غير الوظيفي الثالث (المعالجة غير المتزامنة) بنجاح، وذلك للأسباب التالية:

- ✓ المهام غير العاجلة (البريد الإلكتروني، الفاتورة) لا تعيق استجابة المستخدم) كما يظهر في (Dashboard)
- ✓ المهام المجدولة والمتكررة تعمل كما هو مخطط لها.
- ✓ النظام يحتفظ بالمهام عند إعادة التشغيل (الاستمرارية).
- ✓ Hangfire Dashboard يوفر مراقبة كاملة وكشفاً عن أي فشل محتمل.
- ✓ ما تظهره الصور من أعداد كبيرة للمهام الناجحة وغياب المهام الفاشلة هو دليل عملي قوي على نجاح تطبيق Hangfire في إدارة المهام غير المتزامنة

## المتطلب غير الوظيفي الرابع:

### معالجة البيانات الضخمة على دفعات (Batch Processing)

#### الجزء الأول: شرح عام عن المشكلة

##### ما هي معالجة البيانات الضخمة على دفعات؟

في أنظمة التجارة الإلكترونية، تتراكم البيانات يومياً بكميات كبيرة: جداول الطلبات، تفاصيل الطلبات، معاملات الدفع، حركة المخزون، وحتى جداول المنتجات قد تصل إلى آلاف أو ملايين السجلات. عند الحاجة إلى إنشاء تقارير مبيعات يومية، تحليل المنتجات الأكثر مبيعاً، أو حتى عرض قائمة المنتجات مع إمكانية التصفح، قد نضطر إلى قراءة كميات هائلة من البيانات من قاعدة البيانات.

المعالجة على دفعات Batch Processing هي تقنية يتم فيها تقسيم مجموعة كبيرة من البيانات إلى أجزاء صغيرة متتالية Chunks ، تُعالج كل منها بشكل منفصل. بدلاً من تحميل جميع السجلات في الذاكرة مرة واحدة والتي قد تستهلك مئات الميجابايت أو حتى الجيجابايت، يتم جلب جزء صغير (مثلاً 500 سجل)، معالجته، ثم التخلص منه، ثم جلب الجزء التالي، وهكذا.

**الهدف:** الحفاظ على استقرار النظام من حيث استهلاك الذاكرة Memory Footprint ووقت الاستجابة، وتجنب انهيار التطبيق بسبب استنفاد الذاكرة أو تجاوز مهلة الاتصال بقاعدة البيانات، وضمان بقاء النظام سريع الاستجابة حتى مع نمو البيانات بشكل كبير.

#### المصطلحات الأساسية

| المصطلح          | المعنى                                                                   |
|------------------|--------------------------------------------------------------------------|
| Batch Processing | معالجة مجموعة من البيانات كوحدة واحدة (دفعة) ضمن مهمة واحدة غير تفاعلية. |

|                                    |                                                                                               |
|------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Chunk</b>                       | جزء صغير من الدفعة الكبيرة، يتم جلبه ومعالجته في كل خطوة.                                     |
| <b>Offset / Pagination</b>         | طريقة لتحديد عدد السجلات التي يتم تخطيها Skip وعددها Take لتقسيم البيانات.                    |
| <b>Cursor-based<br/>Pagination</b> | طريقة أكثر كفاءة من Offset ، تعتمد على استخدام قيمة فريدة مثل Id لتحديد نقطة البداية التالية. |
| <b>Out-of-memory (OOM)</b>         | خطأ يحدث عند محاولة تخصيص ذاكرة أكبر مما هو متاح للتطبيق.                                     |
| <b>Long-running query</b>          | استعلام يستغرق وقتاً طويلاً (أكثر من 30 ثانية) وقد يتسبب في انتهاء المهلة أو قفل الجداول.     |

## الجزء الثاني: شرح المشكلة في مشروعنا

### السيناريو الأول: تقرير المبيعات اليومية

في نظام التجارة الإلكترونية، من المتطلبات الوظيفية:

- عرض إجمالي المبيعات لليوم الحالي.
- عرض المنتج الأكثر مبيعاً خلال آخر 7 أيام.

لحساب هذه الإحصائيات يدوياً (بدون دفعات)، قد نكتب كوداً مثل:

```
var orders = await _context.Orders
    .Where(o => o.OrderDate >= startDate && o.OrderDate < endDate)
    .Include(o => o.OrderItems)
    .ToListAsync(); // تحميل جميع الطلبات في الذاكرة دفعة واحدة
```

إذا كان عدد الطلبات في اليوم الواحد 100,000 طلب، وكل طلب يحتوي على 5-10 عناصر (OrderItems)، فإن كمية البيانات المسترجعة ستكون ضخمة جداً. قد يصل حجم البيانات إلى عدة مئات من الميجابايتات، مما يؤدي إلى:

- استهلاك كبير للذاكرة على الخادم قد يصل إلى OutOfMemoryException .
- تباطؤ شديد في معالجة الاستعلام وجلب البيانات.
- انتهاء المهلة (Timeout) إذا استغرق الاستعلام أكثر من 30 ثانية.
- تأثير سلبي على الأداء العام للنظام، حيث أن عملية التقرير تستهلك موارد قاعدة البيانات وتؤثر على عمليات الشراء المتزامنة.

### السيناريو الثاني: عرض قائمة المنتجات

عندما ينمو عدد المنتجات إلى عشرات أو مئات الآلاف (كما هو الحال في المتاجر الإلكترونية الكبيرة)، فإن جلب جميع المنتجات دفعة واحدة في طلب `GET /api/products` سيؤدي إلى:

- إرسال كمية ضخمة من البيانات عبر الشبكة .
- استهلاك كبير للذاكرة على الخادم والعميل.
- زمن استجابة طويل جداً للمستخدم، خاصة على الأجهزة المحمولة أو الاتصالات البطيئة.

### لماذا هذه المشكلة خطيرة في نظام تجارة إلكترونية؟

- تراكم البيانات: مع الوقت، يزداد عدد الطلبات والمنتجات بشكل خطي (أو أسّي).
- اليوم الأول: 100 طلب، بعد سنة: 100,000 طلب/يوم. عدد المنتجات قد يتجاوز 100,000 خلال سنوات قليلة.
- الذاكرة محدودة : خادم التطبيق غالباً لديه 4-8 جيجابايت RAM. تحميل 500,000 سجل في الذاكرة غير ممكن.
- الاستعلامات الطويلة: عندما تكون الجداول مفهرسة بشكل غير مناسب، أو الاستعلامات معقدة، قد تنهار قاعدة البيانات.

لذلك، يجب اعتماد معالجة دفعات **Batch Processing** في المهام التي تتعامل مع كميات كبيرة من البيانات، سواء كانت تقارير دورية أو واجهات عرض للمستخدم النهائي.

## الجزء الثالث: الحلول الممكنة

### الحل الأول: استخدام Skip و Take

**الفكرة:** جلب البيانات على شكل صفحات: نحدد حجم الصفحة Page Size ونستخدم Skip لتخطي عدد من السجلات و Take لجلب عدد محدد.

```
int pageSize = 500;int pageSize = 500;
int page = 0;
List<Order> batch;

do
{
    batch = await _context.Orders
        .OrderBy(o => o.Id)
        .Skip(page * pageSize)
        .Take(pageSize)
        .ToListAsync();

    ProcessBatch(batch);
    page++;
} while (batch.Count == pageSize);
```

#### المميزات:

- بسيط وسهل الفهم والتطبيق.
- مدعوم في EF Core و SQL Server.

#### العيوب:

- ضعيف الأداء على الصفحات الكبيرة: في كل مرة نستخدم Skip، يقوم SQL Server بمسح جميع الصفوف السابقة. مثلاً Take(100000)Skip(500) سيقراً 100,500 صف ثم يتجاهل أول 100,000 صف. هذا يؤدي إلى بطء شديد في الصفحات المتأخرة.
- غير مناسب للبيانات التي تتغير أثناء المعالجة (إذا أضيفت طلبات جديدة أثناء التقرير، قد تتكرر بعض السجلات أو تُفقد).



## الحل الثاني: استخدام Cursor-based Pagination

**الفكرة:** استخدام عمود فريد وقابل للفرز (مثل Id المتزايد) لتحديد نقطة البداية التالية. بدلاً من Skip، نضيف شرط WHERE Id > lastId

```
int batchSize = 500;
int lastId = 0;
List<Order> batch;

do
{
    batch = await _context.Orders
        .Where(o => o.Id > lastId)
        .OrderBy(o => o.Id)
        .Take(batchSize)
        .ToListAsync();

    ProcessBatch(batch);
    if (batch.Any())
        lastId = batch.Last().Id;
} while (batch.Count == batchSize);
```

### المميزات:

1. أداء ممتاز بغض النظر عن حجم البيانات .
2. لا يتأثر بإضافة سجلات جديدة أثناء المعالجة.
3. مناسب جداً للبيانات المتسلسلة والمتزايدة.

### العيوب:

- يتطلب وجود عمود متزايد فريد. إذا كان مفتاح التصفية غير متزايد (مثل التاريخ)، قد لا يعمل بكفاءة.
- غير مناسب للتصفية حسب حقول أخرى غير متسلسلة.

## الحل الثالث: استخدام Hangfire Recurring Jobs مع Batch Processing

**الفكرة:** جدولة وظيفة Hangfire لتشغيلها يومياً (الساعة 2 صباحاً مثلاً) وتنفيذ عملية تجميع إحصاءات المبيعات على دفعات Chunks باستخدام Cursor-based Pagination.

### المميزات:

- المهمة تعمل في الخلفية، لا تؤثر على أداء واجهة API.
- يمكن إعادة محاولتها إذا فشلت.
- يمكن مراقبتها عبر Hangfire Dashboard.
- يمكن توزيعها عبر خوادم متعددة.

### العيوب:

- يتطلب تكوين Hangfire.
- يحتاج إلى كتابة منطق الدفعات Chunking يدوياً.

## الحل الرابع: تطبيق Pagination على مستوى واجهة API

**الفكرة:** تعديل نقطة نهاية GET /api/products لقبول معاملات page و pageSize أو cursor و limit، وإرجاع دفعة صغيرة من المنتجات بدلاً من القائمة الكاملة. يمكن استخدام Skip/Take أو Cursor Pagination حسب الحجم المتوقع.

## الجزء الرابع: مقارنة الحلول

| الحل                                            | الصعوبة | الأداء على كميات كبيرة          | استهلاك الذاكرة                | التوافق مع EF Core | متطلبات إضافية   |
|-------------------------------------------------|---------|---------------------------------|--------------------------------|--------------------|------------------|
| <b>Skip/Take</b>                                | سهلة    | ضعيف<br>(خاصة الصفحات المتأخرة) | منخفض<br>(بسبب التحميل الدفعي) | ممتاز              | لا شيء           |
| <b>Cursor<br/>Pagination<br/>(Id&gt;lastId)</b> | متوسطة  | ممتاز<br>(باستخدام فهرس)        | منخفض                          | جيد (LINQ)         | عمود متزايد فريد |
| <b>Server-side<br/>Cursor</b>                   | صعبة    | ممتاز                           | جداً منخفض<br>(صف صف)          | ضعيف<br>(خام SQL)  | لا شيء           |
| <b>Hangfire +<br/>Cursor<br/>Pagination</b>     | متوسطة  | ممتاز                           | منخفض                          | ممتاز              | Hangfire         |

## الجزء الخامس: الحل الأنسب لمشروعنا مع الأسباب

تم اختيار حلين مكملين لتحقيق المتطلب الرابع، حسب نوع العملية:

🚦 للتقارير Hangfire Recurring Job مع Cursor-based Pagination لحساب إحصاءات المبيعات اليومية وتخزين النتيجة في جدول منفصل SalesDailySummary.

## لماذا هذا المزيج؟

- استخدام Hangfire لتشغيل المهمة يومياً في وقت منخفض الضغط (مثلاً الساعة 2 صباحاً) دون تأثير على المستخدمين.
  - استخدام Cursor Pagination (`WHERE Id > lastId`) لضمان قراءة الطلبات على دفعات صغيرة (حجم الدفعة مثلاً 500 طلب) مما يحافظ على الذاكرة والأداء.
  - نتائج محسوبة مسبقاً: بدلاً من حساب الإحصائيات عند الطلب، تُحسب مرة واحدة يومياً وتُخزن في جدول SalesDailySummary. عند عرض التقرير، يقرأ النظام القيم الجاهزة مباشرة استعلام خفيف وسريع.
  - استمرارية: إذا فشلت المهمة لأي سبب (انقطاع قاعدة البيانات، خطأ مؤقت)، يعيد Hangfire المحاولة تلقائياً.
- 🚦 لعرض المنتجات Cursor-based Pagination : على مستوى واجهة API بدون مهمة خلفية.

## لماذا هذا الحل؟

- أداء ثابت بغض النظر عن عدد المنتجات الكلي، حيث يستخدم فهرس المفتاح الأساسي .
- تجنب تحميل المنتجات غير المرئية للمستخدم: المستخدم لا يحتاج إلى رؤية 100,000 منتج دفعة واحدة؛ الدفعات الصغيرة مثلاً 20 منتجاً تكفي.
- مرونة التنقل: باستخدام cursor (آخر `Id` تم استلامه)، يمكن للعميل طلب الصفحة التالية بسهولة دون إعادة حساب إزاحة ضخمة.
- قابلية التوسع: حتى مع نمو عدد المنتجات إلى ملايين، يبقى زمن استجابة الطلب لكل دفعة ثابتاً.

## الجزء السادس: التطبيق العملي

### تطبيق Batch Processing على التقارير

إنشاء خدمة Batch Processing

```
public interface ISalesSummaryBatchService
{
    Task ProcessDailySalesSummaryAsync(DateTime targetDate);
}

public class SalesSummaryBatchService : ISalesSummaryBatchService
{
    private readonly AppDbContext _context;
    private readonly ILogger<SalesSummaryBatchService> _logger;

    public SalesSummaryBatchService(AppDbContext context,
    ILogger<SalesSummaryBatchService> logger)
    {
        _context = context;
        _logger = logger;
    }

    public async Task ProcessDailySalesSummaryAsync(DateTime targetDate)
    {
        var startDate = targetDate.Date;
        var endDate = startDate.AddDays(1);

        _logger.LogInformation($"بدء معالجة دفعات المبيعات لليوم {startDate:yyyy-MM-dd}");

        const int batchSize = 500;
        int lastOrderId = 0;
        int totalOrders = 0;
        decimal totalSales = 0;
        var productSales = new Dictionary<int, int>(); // ProductId ->
        TotalQuantitySold

        bool hasMore = true;
        while (hasMore)
        {
            var ordersBatch = await _context.Orders
                .Where(o => o.OrderDate >= startDate && o.OrderDate < endDate &&
                o.Id > lastOrderId)
                .OrderBy(o => o.Id)
                .Take(batchSize)
                .Include(o => o.OrderItems)
                .ToListAsync();
```

```

        if (!ordersBatch.Any())
        {
            hasMore = false;
            break;
        }

        foreach (var order in ordersBatch)
        {
            totalOrders++;
            totalSales += order.TotalAmount;

            foreach (var item in order.OrderItems)
            {
                if (productSales.ContainsKey(item.ProductId))
                    productSales[item.ProductId] += item.Quantity;
                else
                    productSales[item.ProductId] = item.Quantity;
            }
        }

        lastOrderId = ordersBatch.Last().Id;
        _logger.LogInformation($"تمت معالجة دفعة {batchSize} آخر، Id = {lastOrderId}");
    }

    int topProductId = 0;
    int maxQuantity = 0;
    foreach (var kvp in productSales)
    {
        if (kvp.Value > maxQuantity)
        {
            maxQuantity = kvp.Value;
            topProductId = kvp.Key;
        }
    }

    var summary = new SalesDailySummary
    {
        Date = startDate,
        TotalOrders = totalOrders,
        TotalSales = totalSales,
        TopProductId = topProductId,
        ProcessedAt = DateTime.UtcNow
    };

    var existing = await _context.SalesDailySummaries
        .FirstOrDefaultAsync(s => s.Date == startDate);
    if (existing != null)
    {
        existing.TotalOrders = totalOrders;
        existing.TotalSales = totalSales;
        existing.TopProductId = topProductId;
        existing.ProcessedAt = DateTime.UtcNow;
    }
}

```

```

else
{
    await _context.SalesDailySummaries.AddAsync(summary);
}await _context.SaveChangesAsync();
_logger.LogInformation($"تم تحديث ملخص المبيعات لليوم {startDate:yyyy-MM-dd}: طلبات = {totalOrders}, مبيعات = {totalSales:C}, أفضل منتج = {topProductId}");
}

```

🚦 تسجيل الخدمة وجدولة المهمة في Program.cs

```
builder.Services.AddScoped<ISalesSummaryBatchService, SalesSummaryBatchService>();
```

// بعد تهيئة Hangfire

```

RecurringJob.AddOrUpdate<ISalesSummaryBatchService>(
    "daily-sales-summary",
    service => service.ProcessDailySalesSummaryAsync(DateTime.UtcNow.Date.AddDays(-
1)),
    Cron.Daily(2, 0));

```

تطبيق Batch Processing على عرض المنتجات

🚦 تعديل ProductsController لاستخدام Cursor Pagination

```

[HttpGet]
public async Task<IActionResult> GetProducts([FromQuery] int cursor = 0,
[FromQuery] int limit = 20)
{
    if (limit > 100) limit = 100;
    if (limit < 1) limit = 1;

    var query = _context.Products
        .Where(p => p.Id > cursor)
        .OrderBy(p => p.Id);

    var products = await query.Take(limit).ToListAsync();

    var nextCursor = products.LastOrDefault()?.Id ?? cursor;
    var hasMore = products.Count == limit;

    return Ok(new
    {

```

```

Data = products,
NextCursor = nextCursor,
HasMore = hasMore
}); }

```

### شرح الآلية:

- العميل يطلب الدفعة الأولى: GET /api/products?limit=10 (cursor=0 افتراضياً).
- الخادم يعيد أول 10 منتجات، ويرسل nextCursor = آخر Id في القائمة (مثلاً 10)، و hasMore = true إذا كان هناك منتجات إضافية.
- الصفحة التالية: GET /api/products?cursor=10&limit=10.
- هذا يضمن أداءً ثابتاً بغض النظر عن العدد الإجمالي للمنتجات، ويجنب مشكلة Skip البطيئة.

## قسم إجراءات الاختبار والإثبات

تم إجراء اختبارات عملية للتحقق من فعالية معالجة الدفعات Batch Processing في جانبين رئيسيين: تقارير المبيعات اليومية و عرض قائمة المنتجات . اعتمدت منهجية الاختبار على الأدوات المتاحة ضمن بيئة التطوير (Swagger، Hangfire Dashboard، قاعدة البيانات) لتكون قابلة للتكرار والتوثيق.

### اختبار Batch Processing على تقارير المبيعات

#### 1.عداد بيانات الاختبار

تم إدراج 2000 طلب تجريبي في قاعدة البيانات داخل Program.cs، مع الحفظ على دفعات كل 500 طلب لتجنب استنزاف الذاكرة أثناء التهيئة..

#### 2.خطوات الاختبار

| الخطوة | الإجراء                             | الأداة                         |
|--------|-------------------------------------|--------------------------------|
| 1      | تشغيل مهمة Hangfire المجدولة يدوياً | Hangfire Dashboard (/hangfire) |



| Recurring Jobs |                                            |                                                     |
|----------------|--------------------------------------------|-----------------------------------------------------|
| 2              | مراقبة حالة المهمة (قيد التنفيذ ثم النجاح) | Processing Hangfire Dashboard تبويب<br>Succeeded ثم |
| 3              | تسجيل زمن استجابة واجهة التقرير            | Swagger + أدوات المطور (F12) – Network tab          |

### التفاصيل:

- **الخطوة 1:** في Hangfire Dashboard ، انتقل إلى **Recurring Jobs** ، ثم اضغط زر **Trigger now** بجانب المهمة `daily-sales-summary` التي تعمل بتعبير `CRON 0 2 * * *` أي الساعة 2 صباحاً يومياً.
  - **الخطوة 2:** بعد تنفيذ المهمة، انتقلت من **Enqueued** إلى **Processing** ثم إلى **Succeeded** بالنقر على المهمة الناجحة، ظهرت التفاصيل التالية:
- Duration – 4m 894ms** الزمن الفعلي لمعالجة جميع الدفعات حوالي 4.9 دقيقة، نتيجة معالجة 2000 طلب مع `OrderItems`
- Latency – 1.504s** وقت انتظار المهمة في قائمة الانتظار قبل بدء التنفيذ.
- Arguments–** التاريخ المستهدف مثلاً "2026-05-09": `"targetDate": "2026-05-09"`
- **الخطوة 3 :** في Swagger ، تم استدعاء `GET /api/reports/daily-summary?date=2026-05-10` باستخدام أدوات المطور (F12) ومراقبة علامة التبويب **Network**، تم تسجيل أقل زمن استجابة للطلب وهو **73 مللي ثانية** ظهر ضمن عدة طلبات، وأسرع وقت كان **73 ms** كما ظهر الرد ببيانات صحيحة

### 3.النتائج المسجلة

| المقياس                     | القيمة المقاسة                                                           | الأداة / المصدر                        |
|-----------------------------|--------------------------------------------------------------------------|----------------------------------------|
| وجود مهمة دورية في Hangfire | مهمة <code>daily-sales-summary</code> بتعبير <code>CRON 0 2 * * *</code> | Hangfire Dashboard<br>(Recurring Jobs) |

|                                                        |                              |                                                  |
|--------------------------------------------------------|------------------------------|--------------------------------------------------|
| <p>تفاصيل Hangfire Dashboard</p> <p>المهمة الناجحة</p> | <p>4m 894ms دقيقة</p>        | <p>زمن تنفيذ مهمة Hangfire</p> <p>ل 2000 طلب</p> |
| <p>أدوات المطور – F12</p> <p>Network tab</p>           | <p>73مللي ثانية أقل قياس</p> | <p>زمن استجابة واجهة التقرير</p>                 |

#### 4. الأدلة المرئية

صفحة **Recurring Jobs** في Hangfire Dashboard تظهر مهمة daily-sales-summary

ما يُثبتها المهمة مجدولة بشكل صحيح ويمكن تشغيلها يدوياً

| Id                  | Cron      | Time zone | Job                                                     | Next execution | Last execution | Created    |
|---------------------|-----------|-----------|---------------------------------------------------------|----------------|----------------|------------|
| daily-sales-summary | 0 2 * * * | UTC       | ISalesSummaryBatchService.ProcessDailySalesSummaryAsync | in 10 hours    | 2 hours ago    | 2 days ago |

الشكل(11)

ما يُثبتته نجاح معالجة الدفعات (الزمن الطويل يعكس حجم البيانات الكبير)

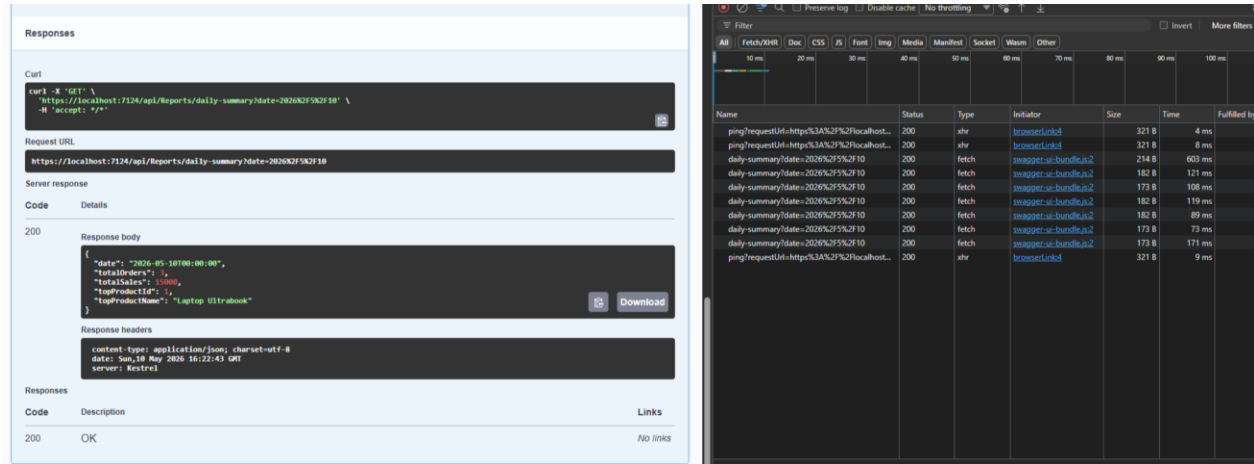
تفاصيل مهمة ناجحة من **Succeeded Jobs** تُظهر Latency، Durations، و Arguments

| Id                  | Cron      | Time zone | Job                                                     | Next execution | Last execution | Created    |
|---------------------|-----------|-----------|---------------------------------------------------------|----------------|----------------|------------|
| daily-sales-summary | 0 2 * * * | UTC       | ISalesSummaryBatchService.ProcessDailySalesSummaryAsync | in 10 hours    | 2 hours ago    | 2 days ago |

الشكل(12)

ما يُثبتته التقرير يقرأ من جدول الملخصات بسرعة فائقة (أقل من 100 مللي ثانية)

استجابة Swagger لنقطة النهاية GET /api/reports/daily-summary مع البيانات الصحيحة، وأدوات المطور تظهر زمن الاستجابة 73 ms



الشكل (13)

## الخلاصة

أثبت الاختبار العملي أن تطبيق معالجة الدفعات (Batch Processing) على التقارير قد حقق:

- فصل الحسابات الثقيلة في مهام دورية خلفية (Hangfire) ، حيث استغرقت مهمة Hangfire حوالي 4.9 دقيقة لمعالجة 2000 طلب .
- تحسيناً هائلاً في زمن استجابة واجهة التقرير، حيث انخفض من عدة ثوانٍ (أو انتهاء مهلة) إلى 73 مللي ثانية فقط، وذلك بفضل قراءة البيانات من جدول SalesDailySummary المُجهَّز مسبقاً.
- موثوقية واستمرارية عبر Hangfire Dashboard الذي يسمح بمراقبة المهمة وإعادة تشغيلها إذا لزم الأمر.

## اختبار Batch Processing على عرض المنتجات

### إعداد بيانات الاختبار

تمت إضافة **100 منتج وهمي** إلى جدول Products ، لضمان تجاوز العدد الافتراضي للدفعة. هذا يتيح اختبار الترحيل Pagination بشكل حقيقي

### خطوات الاختبار والنتائج

| الخطوة | الإجراء                                                                  | الأداة               | النتيجة                                                                                                       |
|--------|--------------------------------------------------------------------------|----------------------|---------------------------------------------------------------------------------------------------------------|
| 1      | استدعاء الدفعة الأولى GET :<br>/api/products?limit=10                    | Postman /<br>Swagger | تم إرجاع 10 منتجات، مع nextCursor=16<br>زمن الاستجابة = <b>281 ms</b>                                         |
| 2      | استدعاء الدفعة الثانية GET :<br>/api/products?cursor=10&limit=10         | Postman              | تم إرجاع المنتجات من Id > 10 من 11 إلى 20<br>تقريباً nextCursor=20 hasMore=true زمن<br>الاستجابة <b>84 ms</b> |
| 3      | استدعاء الدفعة الثالثة GET :<br>/api/products?cursor=20&limit=10         | Postman              | تم إرجاع المنتجات من Id > 20 من 21 إلى 30<br>hasMore=true nextCursor=30<br>الاستجابة <b>84 ms</b>             |
| 4      | محاولة تجاوز الحد الأقصى GET :<br>/api/products?limit=300                | Postman              | تم إرجاع 100 منتج كحد أقصى لأن الكود يقيد<br>hasMore=true nextCursor=110 limit<br>زمن الاستجابة <b>52 ms</b>  |
| 5      | استدعاء طلب كبير مع<br>cursor: GET<br>/api/products?cursor=100&limit=400 | Postman              | تم إرجاع 100 منتج Id من 101 إلى 200<br>تقريباً nextCursor≈200 hasMore=true<br>الاستجابة <b>52 ms</b>          |

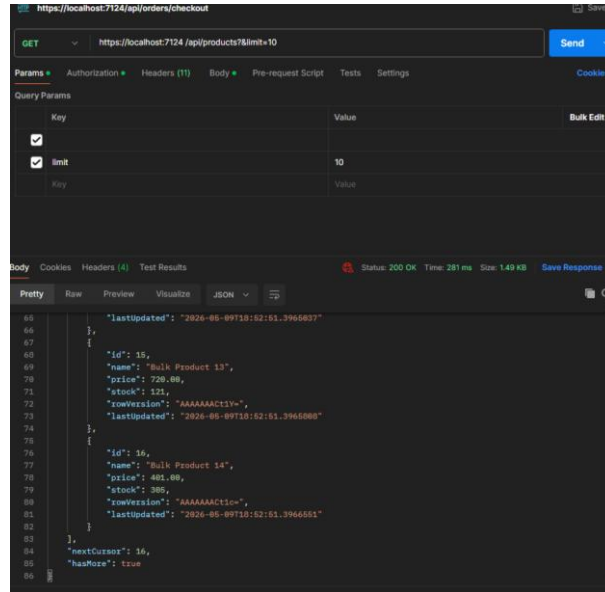
## ملاحظات هامة من التجارب:

- الحد الأقصى للـ **limit** تم تطبيقه بنجاح **limit** لا يتجاوز 100. عند إرسال 300 أو 400، استجاب الخادم بـ 100 منتج فقط.
- زمن الاستجابة لا يعتمد على حجم الدفعة بشكل كبير 84-52: مللي ثانية، والأهم أنه لا يتدهور مع زيادة **cursor** بفضل استخدام **Id > cursor**
- **hasMore: true** في جميع الردود يشير إلى وجود منتجات أكثر (أكثر من 200 منتج)، مؤكداً أن **Pagination** تعمل بشكل صحيح

## الأدلة المرئية

ما يُثبتهُ جلب دفعة صغيرة فقط، وليس كل المنتجات

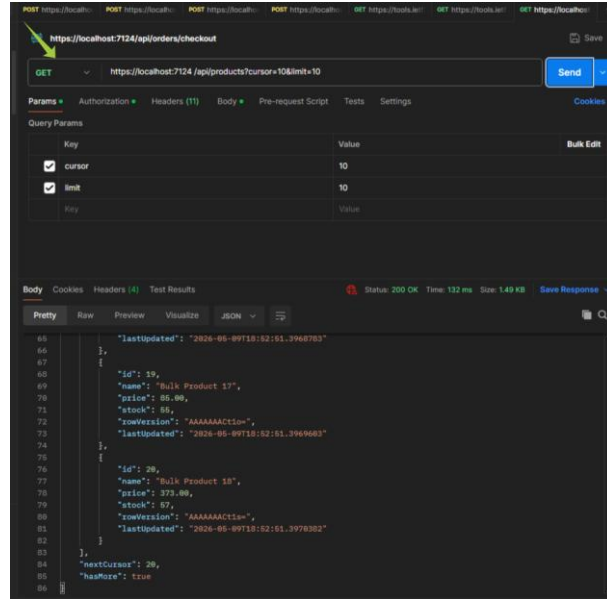
طلب `GET /api/products?limit=10` يعيد 10 منتجات مع `nextCursor=16`، `hasMore=true`، زمن 281ms



الشكل(14)

ما يُثبتهُ التنقل بين الدفعات يعمل باستخدام **cursor** ، والزمن أقل من الدفعة الأولى

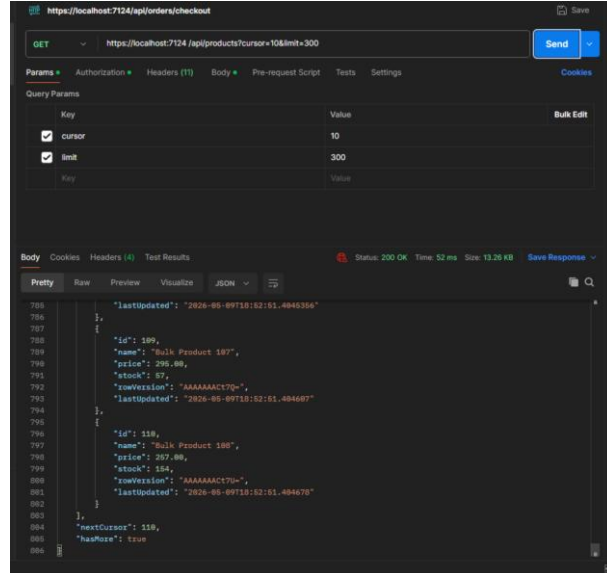
طلب `GET /api/products?cursor=10&limit=10` يعيد المنتجات من 11 إلى 20 مع `nextCursor=20`، زمن 84



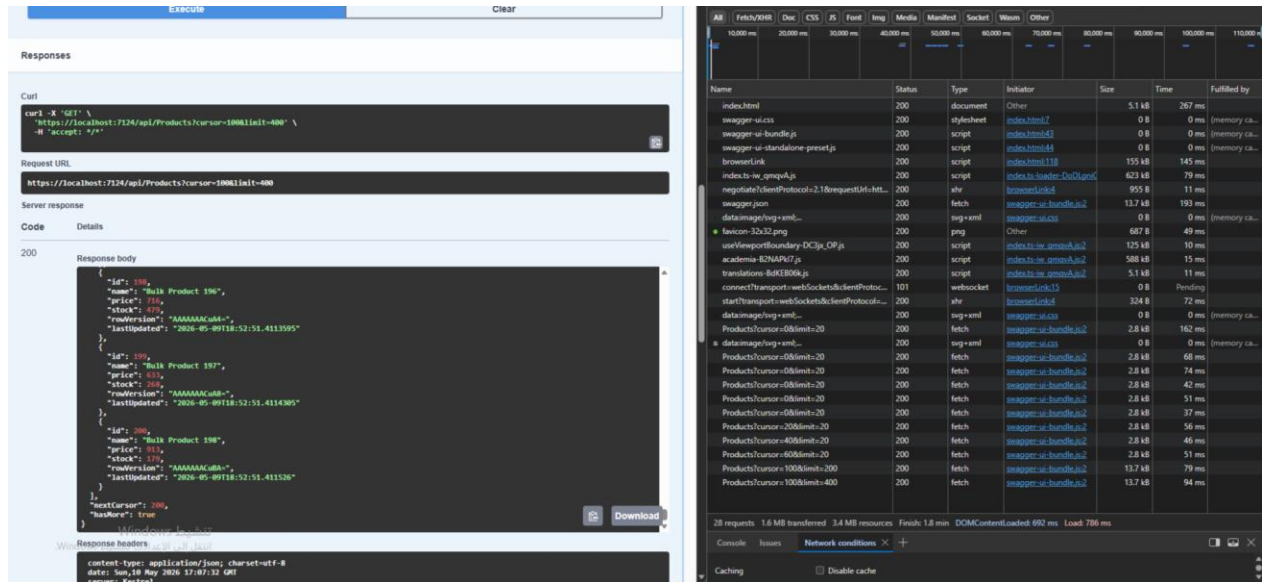
الشكل (15)

ما يُثبت تطبيق الحد الأقصى لحجم الدفعة (100) يحمي الخادم من الطلبات الضخمة

طلب `limit=300` (يتجاوز الحد) يعيد 100 منتج كحد أقصى مع `nextCursor=110`، زمن `ms52`



الشكل (16)



الشكل (17)

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (10ms) [Parameters=[@__p_1='?' (DbType = Int32), @__cursor_0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
  SELECT TOP(@__p_1) [p].[Id], [p].[LastUpdated], [p].[Name], [p].[Price], [p].[RowVersion], [p].[Stock]
  FROM [Products] AS [p]
  WHERE [p].[Id] > @__cursor_0
  ORDER BY [p].[Id]
```

الشكل (18)

## تحليل النتائج

- ثبات الأداء: زمن الاستجابة لـ `cursor=20` (84 ms) مماثل لـ `cursor=100` (52 ms) هذا يثبت أن **Cursor Pagination** لا يعاني من تباطؤ في الصفحات المتأخرة، بعكس `Skip/Take` الذي يزداد سوءاً كلما زاد رقم الصفحة.
- تطبيق القيود: الخادم يرفض تلقائياً `limit` أكبر من 100 ويعيد 100 منتج كحد أقصى. هذا يمنع استهلاك الذاكرة المفرط ويحمي النظام.
- التنقل السلس: إرجاع `nextCursor` و `hasMore` يمكن العميل من التنقل بسهولة دون الحاجة إلى معرفة العدد الإجمالي للمنتجات.
- كفاءة قاعدة البيانات: الاستعلام `(WHERE Id > cursor)` يستخدم فهرس المفتاح الأساسي، مما يجعله سريعاً جداً حتى مع ملايين الصفوف.

## خلاصة الاختبار

أثبتت الاختبارات العملية أن تطبيق معالجة الدفعات (Batch Processing) على عرض المنتجات حقق:

- ثباتاً في زمن الاستجابة بغض النظر عن cursor، وذلك بفضل Cursor Pagination القائم على WHERE Id > cursor
- حماية الخادم من خلال تقييد limit بقيمة قصوى (100)، مما يمنع استنزاف الذاكرة أو استلام دفعات ضخمة.
- كفاءة عالية في قاعدة البيانات باستخدام فهرس المفتاح الأساسي بدلاً من OFFSET الذي يبطئ كلما زاد حجم البيانات.
- تجربة مستخدم سلسة مع إرجاع nextCursor و hasMore لتمكين التنقل دون تعقيد.



# المتطلب غير الوظيفي الخامس: توزيع الأحمال (Load Distribution)

## الجزء الأول: شرح عام عن المفهوم

### ما هو توزيع الأحمال؟

في عالم الخوادم والتطبيقات عالية الضغط، لا يمكن لأي خادم واحد أن يتحمل آلاف الطلبات المتزامنة لفترة طويلة دون أن يتأثر أدائه أو ينهار. توزيع الأحمال Load Distribution أو Load Balancing هو تقنية يتم فيها توزيع الطلبات الواردة من المستخدمين على مجموعة من الخوادم المتطابقة بدلاً من خادم واحد. يتم ذلك عبر موزع أحمال (Load Balancer)، وهو مكون قد يكون برنامجاً أو جهازاً مادياً) يستقبل كل الطلبات، ثم يقرر أي خادم سيتولى معالجة كل طلب وفقاً لاستراتيجية محددة.

### الهدف الرئيسي:

- زيادة الإنتاجية (Throughput): بمعالجة المزيد من الطلبات في نفس الوقت.
- تحسين التوفرية (High Availability): إذا تعطل أحد الخوادم، يستمر النظام في العمل عبر الخوادم الأخرى.
- تقليل زمن الاستجابة (Latency): بتوزيع الحمل، لا يثقل أي خادم بمفرده.
- تجنب الفشل المتتالي (Cascading Failure): لا يؤدي انهيار خادم واحد إلى انهيار النظام بأكمله.

في سياق مشروعنا، مع أننا نعمل على خادم واحد حالياً، يجب محاكاة توزيع الأحمال لإثبات أن نظامنا قادر على العمل في بيئة إنتاجية حقيقية ذات خوادم متعددة. سنستخدم أداة برمجية تسمى YARP (Yet Another Reverse Proxy) من Microsoft كموزع أحمال.

## الجزء الثاني: شرح المشكلة في مشروعنا

### السيناريو الحالي (خادم واحد)

حالياً، كل الطلبات تذهب مباشرة إلى خادم واحد فقط (<https://localhost:7124>). هذا يعني:

- نقطة فشل واحدة (Single Point of Failure): إذا تعطل هذا الخادم، يصبح النظام بأكمله غير متاح.
- محدودية الإنتاجية: خادم واحد له حدود في عدد الخيوط المتزامنة، الذاكرة، واستخدام المعالج.
- لا توجد مرونة للتوسع: لا يمكننا ببساطة إضافة خادم آخر دون تعديل الكود.

ولكن في بيئة إنتاجية حقيقية، يكون هناك عدة خوادم متطابقة تعمل خلف موزع أحمال (مثل Azure Load Balancer، AWS ELB، Nginx، أو YARP). موزع الأحمال يستقبل الطلبات من الإنترنت، ثم يوزعها على الخوادم المتاحة.

المشكلة: مشروعنا التجريبي حالياً لا يعكس هذه البنية. نحتاج إلى محاكاة هذا السلوك بأدوات بسيطة ومجانية لتوضيح فهمنا للمفهوم.

## الجزء الثالث: الحلول الممكنة لمحاكاة توزيع الأحمال

### الحل الأول: استخدام Nginx

الفكرة: تثبيت Nginx محلياً وتكوينه كـ reverse proxy لتوزيع الطلبات بين عدة نسخ من تطبيق .NET. تعمل على منافذ مختلفة.

**المميزات:** أداء عالٍ، شهرة واسعة، يدعم استراتيجيات توزيع متعددة.

**العيوب:** يتطلب تثبيت برنامج خارجي، تهيئة يدوية معقدة نسبياً، لا يتكامل مباشرة مع مشروع .NET.

الحل الثاني: استخدام Docker Compose مع traefik أو nginx

**الفكرة:** تشغيل عدة containers من تطبيق API و container من موزع الأحمال كخدمة Docker.

**المميزات:** قابلية التوسع من خلال سطر أوامر، تكرار البيئة بسهولة.

**العيوب:** يتطلب معرفة بـ Docker و Docker Compose، قد يكون مبالغاً لمشروع بسيط.

الحل الثالث: استخدام YARP من Microsoft

الفكرة: YARP هي مكتبة .NET مفتوحة المصدر تسمح ببناء وكيل عكسي Reverse Proxy مخصص داخل تطبيق ASP.NET Core نفسه. يمكننا تشغيل مشروع YARP مستقلاً يقوم بتوزيع الطلبات على عدة نسخ من مشروعنا الأساسي.

**المميزات:**

- مكتوب بـ .NET، يتكامل بسلاسة مع مشاريعنا.
- سهل التهيئة عبر appsettings.json
- يدعم استراتيجيات توزيع متعددة (Round Robin، Least Requests، Power of Two Choices، إلخ).
- يمكن توسيعه برمجياً حسب الحاجة.

## الحل الرابع: استخدام Azure Load Balancer أو AWS ELB

**الفكرة:** نشر التطبيق على سحابة عامة واستخدام خدمة موازنة الأحمال المدارة.

**المميزات:** إدارة كاملة، ميزات متقدمة .

**العيوب:** يتطلب حساب سحابي ودفع رسوم .

## الجزء الرابع: مقارنة الحلول

| الحل                            | الصعوبة                     | متطلبات إضافية   | مرونة التوزيع         | مناسب لمشروع محلي       | منصة                |
|---------------------------------|-----------------------------|------------------|-----------------------|-------------------------|---------------------|
| <b>Nginx</b>                    | متوسطة/عالية                | تثبيت Nginx      | عالية                 | نعم (لكن تعقيد التثبيت) | Windows/Linux       |
| <b>Docker Compose + Traefik</b> | عالية                       | Docker Desktop   | عالية                 | نعم (لكن تعقيد docker)  | Windows/Linux       |
| <b>YARP</b>                     | منخفضة (بعد الإعداد الأولي) | .NET SDK فقط     | عالية (قابلة للبرمجة) | ممتاز (موصى به)         | Windows/Linux/macOS |
| <b>خدمات سحابية</b>             | منخفضة (توجيه)              | حساب سحابي / دفع | عالية                 | لا (مشروع خارجي)        | سحابي               |

**القرار:** نختار YARP لأنه:

- مبني على .NET، نفس تقنية مشروعنا.
- سهل التثبيت والتشغيل (أمر dotnet new ثم تعديل ملف التكوين).

- يمكننا تشغيله على نفس الجهاز مع النسخ المتعددة من API.
- يوضح المفهوم دون الحاجة إلى أدوات خارجية معقدة.

## الجزء الخامس: الحل الأنسب للمشروع

سنقوم بـ:

1. إنشاء مشروع YARP جديد ك Reverse Proxy.
2. تعديل إعدادات appsettings.json لمشروع YARP لتوزيع الطلبات إلى نموذجين (3 نسخ) من تطبيق API الأصلي تعمل على منافذ مختلفة: 5000, 5010, 5020 (مثلاً).
3. تشغيل النسخ الثلاث من API. على المنافذ المحددة.
4. تشغيل مشروع YARP على المنفذ 8080 (مثلاً) كمدخل موحد.
5. إرسال طلبات الاختبار إلى <http://localhost:8080> ومراقبة التوزيع.

لماذا 3 نسخ؟ لأن موزع الأحمال يظهر فائدته عند وجود أكثر من خادم خلفه. ثلاث نسخ كافية لإيضاح المفهوم.

## الجزء السادس: التطبيق العملي إعداد YARP

## الخطوة 1: إنشاء مشروع YARP

افتح cmd أو terminal في مجلد الحل الرئيسي :

```
dotnet new web -n ECommerceBack.YarpProxy
dotnet sln add ECommerceBack.YarpProxy/ECommerceBack.YarpProxy.csproj
cd ECommerceBack.YarpProxy
dotnet add package Yarp.ReverseProxy -v 2.2.0
```

...

## الخطوة 2: تعديل Program.cs لمشروع YARP

استبدل المحتوى بـ:

```
using Yarp.ReverseProxy;using Yarp.ReverseProxy;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));
var app = builder.Build();
app.MapReverseProxy();
app.Run();
```

## الخطوة 3: إضافة إعدادات موازن الأحمال في appsettings.json

استبدل محتوى appsettings.json بـ:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Yarp": "Debug"
    }
  },
  "AllowedHosts": "*",
  "ReverseProxy": {
    "Routes": {
      "ecommerce-route": {
        "ClusterId": "ecommerce-cluster",
        "Match": {
          "Path": "{**catch-all}"
        }
      }
    }
  },
  "Clusters": {
    "ecommerce-cluster": {
      "LoadBalancingPolicy": "RoundRobin",
      "Destinations": {
        "instance1": {
```

```
{
  "Address": "https://localhost:5000/"
},
{
  "Address": "https://localhost:5010/"
},
{
  "Address": "https://localhost:5020/"
}
}
```

### شرح الإعدادات:

- **Routes** : يحدد أن أي مسار ({catch-all}) يتم توجيهه إلى ecommerce-cluster.
- **Clusters**: مجموعة الخوادم الوجهة. هنا 3 نسخ على المنافذ 5000، 5010، 5020.
- **LoadBalancingPolicy**: استراتيجية التوزيع RoundRobin (أي طلب بالتناوب بين الخوادم). يمكن تغييرها إلى LeastRequests أو PowerOfTwoChoices حسب الحاجة.

#### الخطوة 4: تعديل إعدادات التشغيل لمشروع API الأصلي

فى Visual Studio:

- أضف 3 ملفات تعريف إطلاق (profiles) جديدة بأسماء Instance1, Instance2, Instance3.
- لكل منها، حدد applicationUrl كما يلي:

Instance1: <https://localhost:5001>; <http://localhost:5000> –

Instance2: <https://localhost:5002>; <http://localhost:5010> –

Instance3: <https://localhost:5003>; <http://localhost:5020> –

## الخطوة 5: تشغيل YARP

من مجلد YarpProxy:.

```
cmd
dotnet run --urls "http://localhost:8080"
```

الآن أصبح لدينا:

- YARP يستمع على http://localhost:8080.

- ثلاثة نسخ من API على المنافذ 5000، 5010، 5020.

- YARP يوزع الطلبات بين النسخ الثلاث بطريقة Round Robin.

## الجزء السابع: اختبار توزيع الأحمال

### اختبار بسيط باستخدام المتصفح

- افتح المتصفح وأدخل http://localhost:8080/api/products.

- قم بتحديث الصفحة عدة مرات وراقب في كل نافذة API (التي تعمل في cmd) أي نسخة استقبلت الطلب.

- ستلاحظ التوزيع بالتناوب: الطلب الأول يذهب إلى 5000، الثاني إلى 5010، الثالث إلى 5020، الرابع إلى 5000، وهكذا.



```
Administrator: موجه الأوامر - dotnet run --urls "http://localhost:8089"
at Yarp.ReverseProxy.Forwarder.HttpForwarder.SendAsync(HttpContext context, String destinationPrefix, HttpMessa
geInvoker httpClient, ForwarderRequestConfig requestConfig, HttpTransformer transformer, CancellationToken cancellatioN
oken)
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5000/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5020/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5010/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5000/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5020/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5010/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5000/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5020/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5010/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5000/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5020/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5010/api/products HTTP/2 RequestVersionOrLower
Info: Yarp.ReverseProxy.Forwarder.HttpForwarder[9]
Proxying to https://localhost:5000/api/products HTTP/2 RequestVersionOrLower
warn: Yarp.ReverseProxy.Forwarder.HttpForwarder[48]
```

الشكل (19)

- [Azure Load Balancer Documentation](#)
- [BackgroundService / IHostedService \(Microsoft Docs\)](#)
- [Backpressure in Reactive Systems](#)
- [Batch Processing in .NET](#)
- [BCrypt.Net-Next NuGet Package](#)
- [BenchmarkDotNet Diagnosers](#)
- [BenchmarkDotNet Documentation](#)
- [BenchmarkDotNet GitHub Repository](#)
- [C# SemaphoreSlim: A Practical Guide](#)
- [Circuit Breaker Pattern](#)
- [Concurrency Control in Database Systems](#)
- [Concurrency Tokens in EF Core](#)
- [Concurrent Collections](#)
- [Connection Pooling in SQL Server](#)
- [Cursor-based Pagination in SQL Server](#)
- [Distributed Locks with Redlock](#)
- [Docker Compose Documentation](#)
- [Efficient Querying with EF Core – Paging](#)
- [Handling Concurrency Conflicts in EF Core](#)
- [Hangfire Background Job Filters](#)
- [Hangfire Background Methods](#)
- [Hangfire Dashboard](#)
- [Hangfire Official Documentation](#)
- [Hangfire Recurring Jobs](#)
- [Hangfire Recurring Jobs](#)
- [Interlocked Class](#)
- [JWT Bearer Authentication](#)
- [Martin Fowler - Unit of Work \(Batch Processing Context\)](#)
- [Message Queuing Patterns](#)
- [Microsoft.AspNetCore.Authentication.JwtBearer NuGet](#)
- [Monitor Class](#)
- [Newtonsoft.Json Documentation](#)
- [Nginx Documentation](#)
- [\\_Optimistic Offline Lock](#)
- [PostgreSQL Pagination Techniques](#)
- [PropertyBuilder.IsRowVersion Method](#)

- [Rate Limiting Middleware in ASP.NET Core](#)
- [Redis Distributed Locks \(RedLock Algorithm\)](#)
- [RedLock.net GitHub Repository](#)
- [RedLock.net NuGet Package](#)
- [Run Multiple Instances of ASP.NET Core Locally](#)
- [SemaphoreSlim Class](#)
- [SKIP LOCKED: Locking for maximum concurrency](#)
- [SQL Server RowVersion](#)
- [StackExchange.Redis Documentation](#)
- [System.Threading.Channels](#)
- [ThreadPool Class](#)
- [Timestamp / RowVersion Attribute](#)
- [YARP \(Yet Another Reverse Proxy\) Official Site](#)
- [YARP Load Balancing \(ASP.NET Core / Microsoft Learn\)](#)
- [YARP Load Balancing Documentation](#)

